Detecting a drop in a count against its own history: one judge at five widths, and what it catches

John Pavlik Claude (Anthropic)

November 9, 2026

Abstract

A source emits a count every five minutes, and an operator needs three things from a detector of drops in it, in order: to know when the source has stopped, to trust the alarm, and to hear of a real degradation as soon as the first two allow. We recommend one detector for all three. It is one judge: a statistic of the source's recent past is held against the empirical quantile of the same statistic over the source's own history, a margin is set beyond the quantile, and the alarm is raised after a wait. The judge is applied at five widths: four count the rows of the last 5 minutes, 30 minutes, 6 hours and 1 day, and the fifth counts the time since the last row. A width judges only where the source gives it enough data for a quantile, so a dense source is judged in rows and a sparse one in time, and every edge is a quantile of the source's own past, with no distribution assumed. On 35 real and modeled sources the detector catches every injected stop, within 20 minutes on a source with rows in every bucket. Against the baseline, false alarms fall from 145 to 0.16 per source-month on the real count series, and a drop to 30% is caught within the day 74% of the time, which the baseline's alarm rate did not allow.

1 Introduction

Operational systems watch counts: rows read from a source, requests served, lines written to a log, readings received from a fleet of sensors. The rule this work replaces, the baseline below, watched them the obvious way: it compared the last 30 minutes with the 91 windows at the same time of day over the past week and alarmed when the current window fell below their 99% band. On real count series it raised about 145 false alarms per source-month, five a day on a healthy source. An alarm that is wrong five times a day is an alarm nobody reads, and a detector nobody reads misses the drop that mattered. That is the problem this paper starts from, and the reason it treats false alarms as the first cost and not an afterthought.

What does an operator need from a drop detector? Three things, in order. First, to know when a source has stopped, on every source, whatever its rate: a source that has gone silent is the one failure that is always a failure. Second, to be able to trust the alarm: a healthy source must almost never alarm, or the first need is not met either, because the stop's alarm is one among the day's false ones. Third, to hear about a real degradation, a source that has fallen to a tenth or a third of its usual rate, as soon as the first two needs allow. The order matters because the three pull against each other: every loosening that catches a shallower drop sooner also lets more healthy days through. This paper takes the order as given and reports where each need is met, where it is paid for, and where it is given up.

Why did the baseline fail, and what fixes it? Its 91 windows overlap in time and share slowly drifting noise, so they hold far fewer independent observations than their number suggests, and a reading at the band's edge flickers in and out, each flicker a new alarm. The repair rests on one distribution-free fact (Appendix A): among n exchangeable windows and one more, the new one

is the lowest at most 1/(n + 1) of the time, and exactly that when no two tie. The fact lets the source's own past draw its band with no distribution assumed, which is what the heavy-tailed counts of real systems need. But the fact alone does not reach one alarm a month. A width judged every five minutes is read 8,640 times a month. An edge at the lowest of a week's two thousand windows is undercut by a healthy source about four of those times by chance, and the design's edge, which sits near the third-lowest, about thirteen. A quantile by itself therefore stops an order of magnitude short of the alarm rate the second need asks for. So the detector pulls three levers the fact leaves open (Section 2). It pools every window of the week, after dividing each by its time of day's typical level, so that two thousand windows stand behind the edge instead of 91. It widens the edge by a margin, so a reading must fall further than any past one did, not merely below the lowest. And it waits for four readings in a row, so a flicker at the edge pays nothing. Each lever is measured on its own in Section 6.1, and together they take the real series from 145 false alarms a month to 0.16.

One judge, then, applied to what? A stop empties a 5-minute window in 5 minutes and a day's window in a day, and a shallow drop that never empties a 5-minute window pulls a day's window far below its usual sum, so no single window sees every drop. The judge is therefore applied at several widths of window, 5 minutes, 30 minutes, 6 hours and 1 day, each with its own quantile over the same history, and a width is judged only where the source's typical window at that width holds ten rows, because a quantile of ratios among a handful of rows means nothing. That one gate is what makes the detector follow the source's density: a dense source is judged at every width and its stop is caught at the fourth empty 5-minute reading, 20 minutes; a source with four rows an hour is judged at 6 hours and 1 day; a source with a row an hour at 1 day alone. The stop is caught as fast as the source's own rows allow and no faster, and no setting of the detector changes that (Section 5.1).

Below a row an hour the count widths run out, and the judge does not. A log that writes a line only when something goes wrong, a service whose rows come in bursts with hours of silence between, a source with one row a day: on these no window up to a day usually holds ten rows, or the history holds empty windows at every time of day and the lower edge is zero, so a count of rows per unit of time can never fall below normal. But every such source has a normal in the other coordinate, time per row: its quiet spells have lengths, and an operator asking whether the source has been silent too long is asking whether the current spell is longer than its spells ever are. So the fifth width is the gap, the time since the last row, judged by the same judge against the source's own completed spells of the past 28 days: the source is stopped when its current silence has outlasted every one of them by half again, and the width is judged only once ten spells have completed, the same gate in the other coordinate (Section 2.8). A dense source has no spells and is judged in rows; a source with one row a day fills no count window and is judged in time; a thin source is judged in both, and a stop is reported by whichever width fires first. Scored on 35 sources, from web servers at hundreds of lines a minute down to a source with one row a day, the detector catches every injected stop, 100% within the day on the real count series and all but one log's within the day (Section 4.8), and the gap raises no false alarm on any count series or sparse source.

The same detector then watches the senders behind a source. A source that gathers the rows of a hundred sensors loses one of them without its total moving by more than the noise, so each sender's own count is judged by the same band, at its quiet setting, and a sender is any value of a field that is seen every hour of a normal week (Sections 2.6 and 4.5).

What is given up is named. A drop to 30% is caught within the hour on a fifth of the real series' injected drops at the default setting, and a halving almost never within the hour; both are caught by the end of the day at the long widths on half to three quarters of them. Sections 6.1 and 6.2 show that no setting of the band buys the halving within the hour without the baseline's alarm rate, and the rest of Section 6 that none of the alternatives tried does either. The trade

is the operator's to make, with the margin and the wait, and the paper gives the frontier they trace.

The headline, on the 14 real count series and the 21 modeled series, is

Real series / modeled series	Rule replaced	Recommended
Full stops caught within 20 minutes	—	57% / 86%
Full stops caught within the hour	86% / 98%	79% / 95%
Full stops caught within the day	—	100% / 100%
False alarms per source-month	145 / 201	0.16 / 0.31
Drops to a tenth caught within the hour	84% / 97%	63% / 82%
Drops to 30% caught within the hour	74% / 88%	19% / 56%
Drops to 30% caught within the day	—	74% / 91%
Halvings caught within the hour	59% / 79%	4% / 34%
Halvings caught within the day	—	52% / 88%

Three things in that table must be read apart, because they behave differently.

Full stops are caught at every setting, because a stop is the one drop every width sees: a count width reads zero, and zero is below any positive lower edge, and the gap grows past any record. Section 4.8 scores the five widths together on every source: every injected stop is caught, 100% within the day on the real count series, 100% on the modeled series and every log but one. The only question is how soon, and the answer is set by the source, not by the margin or the wait: the narrowest count width whose windows usually hold rows catches the stop at its fourth empty reading, 20 minutes on a source with rows in every 5-minute bucket, 45 minutes where the 30-minute width is the narrowest judged; below that the gap takes over, and a source with a dozen rows an hour is confirmed stopped in about an hour, one with four rows an hour in about 3 hours, one with a row an hour in about 10 hours with the count widths beside it, 11 by the gap alone (Sections 4.4 and 4.8). Every stop on every real series is caught within the day. A source whose rows come in bursts with silences between is the gap's alone, which asks only whether the current quiet spell has outlasted every normal one by a margin (Section 4.7).

Degradations trade against false alarms. A drop that leaves rows in the window is caught only when it pulls the window under the band's lower edge, and where that edge sits is what the margin and the wait buy. Section 5 is about that trade: which series pay it, why, and what each loosening buys back. The baseline's 74% of drops to 30% rests on five false alarms a day per source, and Section 5.3 argues that at that rate its detection is on paper only.

Time is the third axis. The 6-hour and 1-day widths catch within their own horizon what no shorter width sees in an hour: a halving does not empty a day's window in an hour, but by the end of the day it has pulled it 50% under, and 52% of halvings on real series are caught by then.

The margin and the wait are the operator's two knobs, and Section 5.2 gives three settings on the frontier they trace: a fast one that catches a stop at 15 minutes for 5 times the false alarms on the real series, the default, and a quiet one that cuts the false alarms again for 5 points of drops to 30%. Section 6 gives the alternatives tried and the reasons they lost. The distribution-free fact that bounds the band's false-alarm rate, and the three levers it leaves, are in Appendix A. The paper follows one implementation, named in Section 8; where a choice comes from it rather than from the problem, such as the sample of rows a sender's series is built from, we say so.

2 The detector

The detector is one judge applied at five widths, and Figure 1 says what each width is: the recipe down the side, the machinery in the cells, a cell that spans the four count widths shared and the gap's column its own. What the cells cannot carry is why. A count is judged as a ratio to its time of day's typical level because the bound of Appendix A asks for exchangeable windows, and a 3 a.m. window and a 3 p.m. window are exchangeable only as ratios. The long widths take 28 days of history because a week holds seven day-long windows that do not overlap, too few for a quantile. The gap pools its spells with no typical level because a record kept per hour of the day left each hour a handful of spells to beat and raised 5 to 17 false alarms per sender-month on a source of a hundred thin senders (Section 6.8). It takes the maximum because 28 days hold tens of spells, not thousands of windows, and the maximum is the quantile tens can support. It waits for one reading because a silence only grows, so a second reading says nothing the first did not, where a count at the band's edge flickers and is asked four times. Sections 2.1 to 2.8 define each row.

One judge, five widths: the recipe down the side, the machinery in the cells



A cell across the four count widths is machinery they share; the gap's column is where it differs. Under exchangeability a count edge is undercut at most $(r + 1)/(n + 1)$ of the time, about 3 in 2,017 at the short widths; the gap's edge at most $1/(n + 1)$ (Appendix A).

One judge, five widths: the recipe down the side, the machinery in the cells. A cell across the four count widths is machinery they share; the gap's column is where it differs.

```
judge(statistic, history, quantile, margin, wait, tail, unlearn):
    at every bucket t where this width is judged:
        edge      = quantile(history(t)), in the statistic's units,
                    moved by margin toward the tail
        beyond(t) = statistic(t) lies past edge on the tail side
        if beyond holds at t and at the wait - 1 readings before it:
            alarm, until a reading is not beyond
            unlearn what the alarm covers

for each count width w in {5 min, 30 min, 6 h, 1 day}:
    judge(statistic = S_w(t),                          # rows in the window
          history   = (S_w + 1) / (M_w + 1) of the learned windows, 7 days (short) or 28 (long),
          quantile  = 0.05%, back in rows as (M_w(t) + 1) q - 1,
          margin = 20% at the default setting, wait = 4, tail = lower,
          unlearn   = the buckets of every window alarmed or waited on,
                      from the one set L the four count widths share)

the gap:
    judge(statistic = g(t),                            # empty buckets since the last row
          history   = lengths of the completed spells of the past 28 days,
          quantile  = the maximum,
          margin = 50%, wait = 1, tail = upper,
          unlearn   = the spell alarmed on)

below normal at t iff some width is alarmed at t
```

Time is discrete in buckets of $\Delta = 5$ minutes; $D = 288$ buckets make a day. A source is a count series $x_t \geq 0$, $t \in \mathbb{Z}$. A window of width w buckets ending at t has sum

$$S_w(t) = \sum_{i=0}^{w-1} x_{t-i}.$$

A reading is one judgment of the source at bucket t. An alarm is raised at t when the decision rule says so and holds until a reading says otherwise; alarms raised, not alarm-buckets, are what an operator sees and what we count. An alarm for a count below its band is a below-normal alarm, above it an above-normal alarm. A source may gather the rows of several senders. When each row names its sender in a field φ with value v, the sender's own count series is x_t^v, the rows at t with $\varphi = v$. In the implementation this series is a sample of about half the rows, so x_t^v is binomially thinned.

2.1 Typical level and typical window

Let $L \subset \mathbb{Z}$ be the learning set, the buckets admitted to learning (Section 2.5). The typical level of a time of day $s \in \{0, \ldots, D-1\}$, as of bucket t, is the median of the learned buckets within 30 minutes of s on each of the past seven days, where $d_D(a, b) = \min((\text{a} - \text{b}) \bmod \text{D}, (\text{b} - \text{a}) \bmod \text{D})$ is the distance between two times of day around the clock,

$$m_t(s) = \text{median}\{\, x_u : u \in L,\ t - 7D \leq u < t,\ d_D(u, s) \leq 6 \,\},$$

about 91 buckets; it is what the charts draw. The typical window of width w ending at u is the median, over the same seven days and the same 30 minutes either side, of the learned window sums ending at that time of day,

$$M_w(u) = \text{median}\{\, S_w(v) : t - 7D \leq v < t,\ d_D(v, u) \leq 6,\ \{v - w + 1, \ldots, v\} \subset L \,\},$$

so that $M_1 = m_t$. A sum of per-bucket medians would be zero on a source whose rows fall in a third of its buckets, while the window that holds them is not (Section 5.4). The profile and the typical windows are refreshed hourly; they move slowly.

2.2 Ratios and band

Every learned window of the past H_w days is expressed as a ratio to the typical level of its time of day, as of t,

$$\rho_w(u) = \frac{S_w(u) + 1}{M_w(u) + 1}, \qquad t - H_w D \le u < t, \quad \{u - w + 1, \ldots, u\} \subset L.$$

The +1 keeps a time of day that is usually empty from dividing by zero and gives it a ratio like any other. Dividing by the typical level is what makes the windows of different days exchangeable enough to pool (Appendix A); the denominators are estimated from the same days as the numerators, so the ratios are exchangeable only approximately. The history H_w is 7 days for the widths up to 30 minutes and 28 days for the longer ones: a week holds about 2,000 distinct 30-minute windows but only 7 distinct days, and the quantile of a width needs enough windows that do not overlap to be a quantile at all (Section 5.4). Let $q_w^-(t)$ and $q_w^+(t)$ be the empirical quantiles of the ratios at levels $(1 - p)/2$ and $(1 + p)/2$, with band percent $p = 99.9\%$, linearly interpolated between the two nearest order statistics, at position $0.0005 \cdot (\mathrm{n} - 1)$ among n ratios: with a week of 30-minute windows, n $\approx$ 2,016, the lower edge lies between the second- and third-lowest ratio, and with 28 days behind a long width, n $\approx$ 8,064, between the fifth- and sixth-lowest, of which only about 28 do not overlap (Appendix A). With margin δ the band at t is

$$\ell_w(t) = (1 - \delta)\left[(M_w(t) + 1)\, q_w^-(t) - 1\right]^+, \qquad h_w(t) = (1 + \delta)\left[(M_w(t) + 1)\, q_w^+(t) - 1\right].$$

The default setting uses $\delta = 0.2$ (Section 2.7). The quantiles are refreshed hourly with the profile. A width is judged only once three days of its history hold learned buckets; before that it has no band.

2.3 Five widths, each where the source fills it

The judge is applied at five widths. Four count rows: w $\in \{1, 6, 72, 288\}$ buckets, the last 5 minutes, 30 minutes, 6 hours and 1 day, each with its own ratios and quantiles over the same learning set. The fifth counts time: the gap g(t), the number of empty buckets since the last row, with its own history of completed spells (Section 2.8). Each width is judged only where the source gives it enough data for a quantile to mean anything. A count width is judged at t when its typical window holds at least B = 10 rows, $M_w(t) \ge \mathrm{B}$, because a window that is usually near empty has a lower edge of zero, and a reading of zero is not below it; the gap is judged when at least ten spells have completed in its history. The gate is what makes the widths follow the source. A source with rows in every bucket is judged at all four count widths and has no spells; a source with a few rows an hour has no 5-minute or 30-minute reading and is judged at 6 hours, 1 day and the gap; a source with a row an hour is judged at 1 day and the gap; a source with a row a day is judged by the gap alone. Each width sees different drops. The 5-minute reading is below its band only when the count is near nothing, which makes it both fast on a stop and silent otherwise. The 30-minute window averages six buckets, so a shallow sustained drop falls below ℓ_6 when it never falls below ℓ_1. The 6-hour and 1-day windows catch a drop to 30% or a halving by the end of the shift or the day, which no short width catches at all. The gap sees one drop, the stop, and sees it on sources where no window does.

2.4 Decision rule

Let $b_w(t) = 1$ if $S_w(t) < \ell_w(t)$ and 0 otherwise, defined only where every bucket of the window is present, a band exists and the width is judged; and let $b_g(t) = 1$ if the current run of empty buckets exceeds the gap's edge of Section 2.8, defined only where the gap is judged. With wait $k = 4$ on the count widths and one reading on the gap, the source is below normal at t when

$$\exists\, w \in \{1, 6, 72, 288\} : \quad b_w(t) = b_w(t-1) = b_w(t-2) = b_w(t-3) = 1, \qquad \text{or} \qquad b_g(t) = 1.$$

The alarm is raised at the first such t; a count alarm ends at the first t at which no count width has four consecutive readings below, a gap alarm at the first row; the decision at t is a function of the stored series alone. The wait is where the two coordinates differ, and for a reason: a count reading at the edge flickers, so four in a row are asked of it, while the gap grows every bucket on its own, so four readings of it in a row say nothing the first did not. A stop is caught at the fourth empty reading of the narrowest judged count width, 20 minutes at 5 minutes, 45 minutes at 30 minutes, within about four hours or a day at the long widths, whose readings are judged every 5 minutes and so fall below their band as soon as the stop has removed enough of the window, and by the gap when the silence has outrun the source's longest normal one by the margin. Above normal is judged at $w = 6$ only, by $S_6(t) > h_6(t)$ four times in a row, and is reported as information: a rise needs no fast detection, and a second width would pay its own false alarms for a message nobody acts on in five minutes.

2.5 Learning set

What an alarm has judged abnormal is not learned as normal, and for the count widths that rule is a set of buckets. A bucket is in L unless it lies under a below-normal count alarm: from the alarm's rise to its end, the buckets of the windows of the widths that fired at each alarmed reading, and the same for the $k - 1$ readings that waited before the alarm was raised. L is one set shared by the four count widths, and a window of any width that contains an unlearned bucket is not learned at that width. A 5-minute alarm that removes an hour of buckets therefore removes the hour from the 5-minute width's history and, from the 1-day width's, every day-long window that overlaps the hour, about 300 of its 8,000; a width that did not fire keeps the rest of its history and its band stands on what remains. Buckets under an above-normal alarm are in L: more is the new normal until proven otherwise. The consequence is an asymmetry in drift. Upward the band follows the source within an hour. Downward it follows only drops the count widths did not alarm on: a reading below ℓ that never reached four in a row is learned, so a shallow drop becomes normal, on a noisy series within 13 hours. A drop the count widths did alarm on is never learned while it lasts, so a dead or halved source stays alarmed until it recovers or is reset.

The gap learns by the same rule in its own coordinate. Its history is the lengths of completed spells, and a spell the gap alarmed on is not learned, so a silence it called a stop never becomes a normal spell. The two histories are separate: a count alarm removes no spell and a gap alarm removes no bucket, so neither coordinate's alarms feed the other's, and a source alarmed in one coordinate goes on learning in the other. An operator's reset of the history empties both.

How an alarm ends differs between the replay and the application, and the difference touches no counted alarm. On a dead source the count widths lose their typical window: M_w is a median over the learned windows of seven days, and once none of them is learned the width has no typical window and no reading, the long widths first, since a day-long window overlaps more unlearned buckets than a 5-minute one. In the replay a count alarm ends when no width judges the source; in the application a below-normal alarm ends only on a reading inside the band or

an operator's reset (Section 8), so a source that has gone quiet stays reported. A gap alarm ends at the first row in both. A false alarm is counted when raised, on stretches where the source is healthy and no width loses its band, and a stop is scored by the delay to its first alarm, so the numbers of Section 4 are the same under either ending.

2.6 Senders

A sender behind a source is a value of a field, and the detector watches a value when its silence would be an event. A value is watched when, over the learning week, it has rows in every hour: its longest run of empty buckets is at most 12. A field is eligible when it has at most 256 distinct values over the week. The cap bounds memory; the hour is what makes a value a sender, since a sender that reports all the time is one whose silence is an event, and a value quiet for an hour somewhere in a normal week would alarm every time it was quiet again. The test is a predicate, not a score: a value is watched or it is not, and a value the test refuses is not judged at all. An operator may name a field to watch or to ignore, may lower one sender's alarm to information when its silence is intended, and a value absent for a whole learning week leaves the watched set. Section 4.5 scores this rule on the fields of real logs and on modeled mixes, and shows why the unit is the value and not the field: nearly every field has one rare value, and a rule that watches a field only when every value passes watches almost nothing.

A watched value is judged by the judge of Sections 2.1 to 2.5 applied unchanged to x_t^v, its own 5-minute series, at the four count widths and at the quiet setting of Section 2.7, because a per-sender false-alarm rate is paid once per sender. The gap is not applied to it: the gap is the width for a source that no count width judges, and a watched value is judged by construction, since rows in every hour put at least 24 in a day's window, so the 1-day width judges it at every hour and the 30-minute width wherever the value is dense enough, which is where its stop is caught within 45 minutes. The cost falls on a sender that is thin at some hour of its day, whose stop at that hour waits for the long widths: on the modeled senders quiet from 1 to 5 a.m., 86% of stops are caught within the hour and all within 6 hours (Section 4.5). The gap on such a value would confirm the stop once the silence outran one and a half times its longest normal spell, 6 hours for a sender quiet from 1 to 5 a.m., no sooner than the long widths; every sender's stop is already caught within 6 hours, and a second width would be a second alarm paid per sender.

2.7 Settings

The margin δ and the wait k are the two knobs, and everything else is fixed. Three settings are measured throughout (Section 5.2):

Setting	k	δ	A stop on a dense source is caught at	False alarms a month, real / modeled
Fast	3	0.2	15 minutes	0.74 / 0.51
Default	4	0.2	20 minutes	0.16 / 0.31
Quiet	4	0.3	20 minutes	0.08 / 0.10

Stops are caught at all three; partial drops and false alarms move together along them. Default names the setting every number not marked otherwise was measured under, and the real logs (Section 4.2), the senders (4.5) and the frontier (5.2) give the quiet setting beside it. The application runs at the quiet setting for sources and senders alike (Section 8): a stop is caught at the same time, the false alarms fall by half on the real series and to a third on the modeled ones, and the three to nine points of degradations it gives up rank last among the operator's needs. Senders are judged at the quiet setting in every case, because a per-sender false-alarm rate is paid once per sender (Section 4.5).

2.8 The gap

The gap is the judge in the coordinate of time per row. Its datum is a quiet spell, a maximal run of empty buckets, measured by its length: a source with a row in most buckets has spells of one or two buckets, a source with a row an hour spells of about twelve, a source with a row a day spells of about three hundred. Its history is the lengths of the spells that completed in the past 28 days and did not lie under one of the gap's own alarms, pooled raw, with no typical level for the time of day. The pooling has a price: on a source with a daily shape the record is a night spell, so a daytime stop is confirmed only once it has outlasted the longest night silence by half again. The price is paid because the alternative was measured and lost: a record kept per time of day leaves each hour a handful of spells to beat, and on a modeled source of a hundred thin senders it raised 5 to 17 false alarms per sender-month, 455 to 1,750 per source-month (Section 6.8). Its quantile is the maximum of the history, L, because 28 days hold tens of spells, not thousands of windows, and Proposition 1 applied to the maximum is the bound: a healthy source whose next spell is exchangeable with the n learned ones outlasts all of them with probability at most $1/(n + 1)$. Its edge is $m{\cdot}L$ with margin $m = 1.5$, upward, since a long gap is the failure where a low count is; the margin is what makes a record implausible rather than merely new, because the longest spell of any 28 days was itself a new record when it arrived, and at $m = 1$ the gap would alarm about once per history length by construction (Section 4.7 measures margins 1, 1.5 and 2). Its gate is ten completed spells, below which the gap is not judged, so a source whose normal silences run a week is judged only once months of spells are in hand. The source is stopped at t when its current run of empty buckets exceeds $m{\cdot}L$, at the first such reading, and the alarm ends at the first row.

What the gap costs and what it cannot do follow from its coordinate. It stores only the list of spell lengths and the start of the current spell, a few thousand integers for a source with rows most hours, a few dozen for one that is silent for days, and none for a dense source, which has no empty bucket and is not judged by it, so its history can be far longer than the count widths' buckets at no cost. What storage cannot buy is speed: a stop is confirmed only when the silence has outlasted the longest normal one by the margin, an hour at a dozen rows an hour, eleven hours at one row an hour, a week or more on a source that is normally silent for days, and no setting changes that, because the source's own habit sets the edge. Nor does the gap see a degradation: a source that halves its rate has longer spells than before, but seldom one longer than its record, so a drop that leaves rows is the count widths' to catch, where they judge. The five widths are judged on every source, each where its gate admits it, and a stop is whichever fires first.

3 How it was measured

Baseline. The rule in use judges $S_6(t)$ against the 99% band of the about 91 windows at the same time of day over the past week, with linear interpolation, alarming at once on a reading outside; windows under an alarm on either side are not learned; judging starts once three days have a window. The implementation has one further trait, reproduced here: an alarm begins a moment after the window that caused it ended, and a window counts as alarmed only if it ends after the alarm began, so the first window of every incident is learned.

Real count series. Fourteen count series from the Numenta Anomaly Benchmark (Lavin and Ahmad, 2015; MIT license) with its labeled incident windows: ten Twitter mention volumes (5-minute counts, 56 days, medians 0 to 50 a bucket, judged from day 29); New York taxi passengers (30-minute counts spread evenly over 5-minute buckets, seven months, judged from day 29); an AWS load balancer's request count and two EC2 network-in series (5-minute, 14 to 17 days, judged from day 8).

Real logs. Ten public logs with a timestamp on every line, counted into 5-minute buckets: four web server access logs of 1994 to 1995 from the Internet Traffic Archive (Arlitt and Williamson, 1996), NASA Kennedy Space Center over two months, ClarkNet over two weeks, the University of Saskatchewan over seven months and the University of Calgary over a year; five system logs from Loghub (He et al., 2020), an OpenSSH server over 28 days, a Zookeeper service over 27 days, a Linux syslog and an Apache error log over 264 days each, and the Blue Gene/L supercomputer's RAS log over 215 days; and the job events of a Google compute cluster over 29 days (Reiss, Wilkes and Hellerstein, 2011). Medians run from 0 to 784 lines a bucket. A run of two hours or more with no line at all, on a log that usually has lines in every bucket, is an outage the archive documents, as NASA's shutdown for a hurricane, or a gap in the collection, and is masked like a labeled incident. Logs of 35 days or more are judged from day 29, shorter ones from day 8. A log's fields, the status code and method of a request, the level and component of a message, the event type of a job, are the fields the sender rule is scored on.

Modeled series. Seven shapes, three seeds each, 56 days: a level, a daily and weekly shape, and multiplicative noise that follows an AR(1) process in log space with about 50 minutes of memory, drawn as Poisson counts. Judged on days 29 to 56.

Shape	Count per 30 min	Noise σ
Steady	1,200	0.1
Variable	1,200	0.6
Daily	360 to 2,040 over the day	0.3
Thin daily	1 to 35 over the day	0.3
Weekday/weekend	Daily on weekdays, a fifth of it at weekends	0.3
Steady, falling 3% a day	1,200, falling	0.1
Variable, falling 3% a day	1,200, falling	0.3

Sparse sources. Four Poisson sources with no daily shape, at 12, 4 and 1 rows an hour and 1 row a day, two seeds each, 84 days judged from day 29, scored at horizons to ten days.

Sender mixes. Eight mixes of 3 to 100 senders behind one source, each a daily shape with AR(1) noise at about 7, 2 or 1 counts a second or 1 a minute; two seeds; 56 days judged from day 29. Each sender's series is binomially thinned by one half, the sample the implementation builds it from (Section 8).

False alarms. Alarms raised per 30 days of judged time outside the labeled incidents; for a sender, per 30 days of that sender's judged time. A bucket is judged when at least one width has a band and is gated in there. An alarm is counted when raised, so a band that flickers pays for every flicker. A rate over a set of series is the mean of the per-series rates, each series weighing the same; we also give the count of alarms and the source-months judged.

Drops detected. At every judged bucket the count is cut to a fraction $f \in \{0, 0.1, 0.3, 0.5\}$ of its level from that bucket on: a stop, a drop to a tenth, a drop to 30%, a halving. The drop is detected within τ if the detector alarms within τ of the cut, following the band as learned up to the cut; we report $\tau = 20$ minutes, 1 hour, 6 hours and 1 day, and the median delay. The cuts are injected at every other judged bucket, so neighboring cuts share the band and the noise and are far from independent; the labeled incidents of the real series serve only to exclude windows from the false-alarm count. The margin and wait were chosen on the real count series (Section 5.2), so their numbers are in-sample; the real logs were added after the band was fixed and are out of sample for it. The gap's margin was chosen with the logs in view (Section 4.7), so for the gap the logs are in-sample too.

Replay. One harness implements the recommended detector as written, its own alarms setting its own learning set; every number in Sections 4 and 5 that is not marked otherwise comes from it. A second implements the baseline, the twelve bands of Section 6.1 and the sweep

of Section 6.2, keeping the baseline's judging and learning rules throughout: windows under an alarm on either side are not learned, and the band is refreshed every reading. Every run is seeded and was reproduced row for row.

4 Results

The results follow the sources from dense to thin, because that is the axis the detector follows. The count series (4.1), the logs (4.2) and the modeled shapes (4.3) show the band where it has rows to judge; the sparse sources (4.4) show where the long widths take over and where even they fall silent; the senders (4.5) show the same band on the values behind a source; each width's own part is separated out (4.6); the gap is scored where the count widths are not (4.7); and the five widths are scored together on every source (4.8), which is where the claim that every stop is caught is made good.

The default setting throughout: widths 5 minutes, 30 minutes, 6 hours and 1 day, 7 and 28 days of history, a 10-row gate, $\delta = 0.2$, $k = 4$. "Months" is the source-months judged: buckets at which at least one width had a band and was gated in. Every share is of the drops injected at judged buckets.

4.1 Real count series

Fourteen series, sorted by how much they carry. "Median" is the median count per 5-minute bucket. False alarms are per source-month, baseline and recommended; the rest is the share of injected drops the recommended detector catches within the horizon.

Series	Median	Months judged	False, baseline	False, recommended	Stops in 20 min	Stops in 1 h	Stops in 6 h	Stops in 1 day	To a tenth in 1 h	To 30% in 1 h	To 30% in 1 day	Halving in 1 day
EC2 network, 257a54	234,246	0.2	239.6	0.0	100%	100%	100%	100%	100%	8%	100%	100%
NYC taxi	2,796	5.5	93.5	0.0	100%	100%	100%	100%	97%	27%	100%	91%
EC2 network, 5abac7	68	0.3	171.2	0.0	93%	93%	97%	100%	93%	27%	100%	93%
Tweets AMZN	50	0.8	265.9	0.0	100%	100%	100%	100%	100%	90%	100%	96%
ELB requests	48	0.2	206.5	0.0	99%	100%	100%	100%	84%	8%	100%	94%
Tweets AAPL	47	0.9	111.6	1.1	100%	100%	100%	100%	89%	38%	87%	54%
Tweets GOOG	16	0.9	138.3	0.0	87%	100%	100%	100%	96%	23%	87%	42%
Tweets FB	14	0.8	131.5	0.0	81%	100%	100%	100%	97%	27%	96%	39%
Tweets KO	8	0.8	196.4	0.0	43%	100%	100%	100%	84%	16%	99%	46%
Tweets IBM	3	0.8	117.9	1.2	0%	88%	100%	100%	14%	1%	46%	17%
Tweets CRM	2	0.9	213.9	0.0	0%	72%	100%	100%	19%	2%	46%	25%
Tweets UPS	2	0.9	109.4	0.0	0%	54%	92%	100%	5%	1%	32%	12%
Tweets CVS	0	0.8	7.1	0.0	0%	0%	36%	100%	0%	0%	8%	4%
Tweets PFE	0	0.9	30.5	0.0	0%	2%	88%	100%	0%	0%	33%	16%

The series fall into three classes. On the nine series with a median of 8 or more a bucket every stop is caught within the hour on eight, 93% on EC2 5abac7, and, on all but Tweets KO, most within 20 minutes; most drops to a tenth are caught within the hour; what is given up within the hour is drops to 30% and halvings, and most of those are caught by the end of the day at the long widths. One of the two real false alarms is here, on Tweets AAPL. On the three series at 2 or 3 a bucket, IBM, CRM and UPS, the 5-minute width is gated out and a stop waits for the 30-minute width: 54% to 88% are caught within the hour and all within the day. The other false alarm is on IBM: Tweets AAPL 1, Tweets IBM 1, 2 alarms in 14.8 source-months.

On the two series with a median of zero, PFE and CVS, the short widths are gated out and only the long ones judge; every stop is caught within the day, where the baseline, judging at 30 minutes only, caught 21% and 3% of their stops within the hour. On PFE and CVS the gap is faster than the long count widths, and a stop is confirmed by one or the other at a median of 3 hours and 6 hours; on IBM, CRM and UPS the 30-minute width is the faster (Section 4.8).

4.2 Real logs

Ten logs the detector was not designed on, sorted by how much they carry; "median" is the median lines per 5-minute bucket, and false alarms are given at all three settings.

Log	Median lines a bucket	Months judged	False, fast / default / quiet	Stops in 20 min	Stops in 1 h	Stops in 6 h	Stops in 1 day	To a tenth in 1 h	To 30% in 1 h	To 30% in 1 day	Halving in 1 day
ClarkNet web, 2 weeks 1995	784	0.2	4.29 / 4.29 / 4.29	100%	100%	100%	100%	100%	61%	100%	57%
Google cluster jobs, 29 days	219	0.7	0.00 / 0.00 / 0.00	100%	100%	100%	100%	98%	50%	100%	85%
NASA web, Jul-Aug 1995	166	1.0	1.03 / 1.03 / 1.03	76%	90%	100%	100%	89%	13%	96%	39%
Saskatchewan web, Jun-Dec 1995	31	6.2	2.91 / 2.26 / 1.29	84%	100%	100%	100%	87%	14%	96%	41%
Calgary web, 1994-95	3	10.8	12.22 / 6.39 / 5.09	1%	42%	93%	100%	2%	1%	56%	29%
OpenSSH, 28 days	2	0.7	4.37 / 2.91 / 2.91	0%	30%	78%	100%	3%	1%	71%	58%
Apache error log, 264 days	0	6.8	7.04 / 6.14 / 5.99	0%	1%	13%	69%	1%	1%	33%	20%
BGL supercomputer, 215 days	0	6.2	17.58 / 17.09 / 17.20	1%	3%	36%	100%	2%	2%	50%	40%
Linux syslog, 264 days	0	6.4	6.86 / 6.55 / 6.85	0%	1%	11%	94%	1%	1%	32%	24%
Zookeeper, 27 days	0	0.0	0.00 / 0.00 / 0.00	0%	0%	0%	0%	0%	0%	0%	0%

Four logs have lines in nearly every bucket. Three of them, ClarkNet, the Google cluster and NASA, behave like the dense count series: every stop within the hour on the first two and 90% on NASA, most drops to a tenth within the hour, and 1, 0 and 1 alarms; ClarkNet's one alarm in 0.2 months is its 4.29 a month. The university servers of 1994 and 1995 drop with the week and the academic calendar: Saskatchewan at 31 lines a bucket raises 2.26 a month and catches every stop within the hour; Calgary at 3 a bucket is judged mostly at the long widths, catches its stops within the day, and raises 6.39 a month, the most of any web log. The four system logs have an empty median bucket, and the band judges them only where a typical 6-hour or 1-day window holds ten lines: every stop is caught within the day on BGL, 94% on Linux and 69% on the Apache error log, whose quiet weeks fill no width, and on those three the false alarms run 6.14 to 17.09 a month, because a log of errors has no normal rate for a ratio to be taken against, and its quietest week sets a band the next quiet week falls under. Some of the Apache alarms stand on silences of 43 to 168 hours, the record's longest completed spells (facts.py), which are stops by any reading and count as false only because the archive does not label them. Zookeeper, 27 days of lines in bursts, fills no width and is not judged. Section 4.7 scores the gap on the same logs and Section 4.8 all five widths together.

4.3 Modeled series

Seven shapes, three seeds each, 2.8 source-months per shape.

Shape	False, baseline	False, recommended	Stops in 20 min	Stops in 1 h	To a tenth in 1 h	To 30% in 1 h	To 30% in 1 day	Halving in 1 h	Halving in 1 day
Steady	146.8	0.00	100%	100%	100%	100%	100%	96%	100%
Steady, falling 3%/day	491.6	0.00	100%	100%	100%	100%	100%	96%	100%
Variable, falling 3%/day	305.6	1.07	100%	100%	100%	80%	100%	21%	100%
Daily	136.7	0.00	100%	100%	100%	72%	100%	14%	100%
Variable	137.2	1.07	100%	100%	81%	17%	100%	4%	82%
Weekday/ weekend	88.8	0.00	100%	100%	40%	20%	40%	5%	37%
Thin daily	102.5	0.00	0%	65%	53%	3%	100%	0%	96%

Where the count is steady the band is tight and everything down to a halving is caught within the hour; the margin costs nothing against a 10% noise. The variable shape, $\sigma = 0.6$ in log space, has windows at a quarter of the typical level inside its own week, so a drop to a tenth is caught and a drop to 30% is not within the hour, and it carries the modeled false alarms: variable seed 2: 1, variable seed 3: 2, variable, falling 3%/day seed 1: 2, variable, falling 3%/day seed 3: 1. The weekday/weekend shape is the one dense modeled series where a drop to a tenth is often missed within the hour: a weekend day sits at a fifth of a weekday, its windows are in the pooled ratios, and the edge sits low; Section 6.7 shows what learning the level per day group buys there. The thin daily shape, 1 to 35 rows a half hour, is gated out of the 5-minute width and, at night, the 30-minute one: its stops and its drops to a tenth wait for a width that holds rows, and 65% of the stops and 53% of the drops to a tenth are caught within the hour.

4.4 Sparse sources

Four Poisson sources with no daily shape, from 12 rows an hour to one a day, 84 days, two seeds. Three detectors: the two short widths alone; the four widths with 7 days of history behind every width; the four widths with 28 days behind the long ones, the design.

Source	Two short widths: false a month	stops in 1 h	stops in 1 day	Four widths, 7-day history: false a month	stops in 6 h	stops in 1 day	Four widths, 28 days behind the long ones: false a month	stops in 6 h	stops in 1 day	median stop
12 rows an hour	not judged			0.0	100%	100%	0.0	100%	100%	3 h
4 rows an hour	not judged			0.0	100%	100%	0.0	100%	100%	4 h
1 row an hour	not judged			0.0	7%	100%	0.0	1%	100%	15 h
1 row a day	not judged			not judged			not judged			—

A source with a dozen rows an hour has 1 row in a typical 5-minute bucket and 6 in a typical half hour, and the short widths are gated out or near it; a source with 4 rows an hour fills only

the 6-hour and 1-day widths, and one with a row an hour the 1-day width alone. With the long widths every stop on the first three is caught, within 6 hours on the first two and within the day on the third, at a median of 3 hours, 4 hours and 15 hours. The 28 days of history behind the long widths cost little here: the source with a row an hour raises 0.0 a month with 7 days behind them and 0.0 with 28, and its stops caught within 6 hours fall from 7% to 1%, the longer history setting the 6-hour edge a little lower. A Poisson source has no shape, and its worst day of seven is near its typical day. The history earns its place on the real series, whose days differ, where 7 days behind every width raised 49 alarms against 2 (Section 5.4). A source with one row a day fills no width up to a day and is not judged by the band; a week-wide width was tried, and the gate admits it only in the weeks that hold ten rows, 0.2 of the 1.9 months the other sparse sources are judged; there it raised 0 alarms and caught a stop after a median of 7 days. Its stop is the gap's to catch (Section 4.7).

4.5 Senders

The detector on each sender's series, as a half sample of its rows sees it, two seeds of each mix:

Senders behind one source	Sender-months	False per sender-month, default / quiet	Stops in 20 min	Stops in 1 h	Stops in 6 h	To a tenth in 1 h	To 30% in 1 h	To 30% in 1 day	Halving in 1 day
10 senders, 7 a second each	18.7	0.59 / 0.00	100%	100%	100%	100%	77%	100%	100%
100 senders, 1 a second each	186.7	0.25 / 0.03	100%	100%	100%	100%	71%	100%	100%
30 small senders, 2 a second	56.0	0.41 / 0.02	100%	100%	100%	100%	74%	100%	100%
3 unequal, 60/30/10% of the rows	5.6	0.00 / 0.00	100%	100%	100%	100%	77%	100%	100%
10 senders in 3 regions, peaks 6 to 8 hours apart	18.7	0.38 / 0.00	100%	100%	100%	100%	76%	100%	100%
10 noisy senders, σ 0.6	18.7	1.50 / 0.75	100%	100%	100%	85%	20%	100%	83%
10 senders quiet from 1 to 5 a.m.	18.7	0.27 / 0.05	83%	86%	100%	76%	58%	100%	100%
100 senders, 1 a minute each	186.7	0.07 / 0.02	0%	66%	100%	30%	1%	100%	99%

Every sender's stop is caught within 6 hours in every mix, and within the hour in every mix but two: the senders quiet from 1 to 5 a.m., whose short widths are gated out at night, and the senders at a row a minute, whose 5-minute width is gated out and whose stop waits for the 30-minute one. Drops to 30% are caught within the day for every sender. The cost is in false alarms at the default setting, 0.43 per sender-month over the eight mixes, two seeds each, 0.00 to 0.59 on the six steady mixes, and a per-sender rate is paid per sender: a source with a hundred senders at 0.25 each is a source that alarms 25 times a month. The quiet setting, $\delta = 0.3$, takes the mean to 0.11 per sender-month, 0.00 to 0.03 on the steady mixes and 0.05 on the mix quiet from 1 to 5 a.m., and leaves the stops where they were, 94% within the hour at both settings; drops to a tenth within the hour go from 86% to 84%. Senders are judged at the quiet setting (Section 2.7).

Which fields of a record identify senders, by the criterion of Section 2.6, on seven modeled fields of one source, three seeds, as a half sample sees them:

Field	Values	Longest silence in the week	Watched
sensor, 10 steady senders	10	none	yes
region, 50/30/20% of rows	3	none	yes
method, 70/25/5% of rows	3	none	yes
status, errors in bursts a few times a day	2	32 to 36 hours	no
employee, 120 people on three shifts, weekdays	120	69 to 91 hours	no
user, 2,000 values	2,000	past the cap	no
build, one value per day	7	6 days	no

The same criterion on the fields of the real logs, over the week before judging starts:

Log	Field	Values	Values seen every hour of the learning week	Field watched
Apache error log, 264 days	level	3	0	no
BGL supercomputer, 215 days	component	14	0	no
BGL supercomputer, 215 days	level	10	0	no
Calgary web, 1994-95	method	3	0	no
Calgary web, 1994-95	status	9	0	no
ClarkNet web, 2 weeks 1995	method	8	2	no
ClarkNet web, 2 weeks 1995	status	7	4	no
Google cluster jobs, 29 days	event	8	4	no
Linux syslog, 264 days	kind	300	0	no
Linux syslog, 264 days	process	70	0	no
NASA web, Jul-Aug 1995	method	11	0	no
NASA web, Jul-Aug 1995	status	8	0	no
OpenSSH, 28 days	kind	28	6	no
OpenSSH, 28 days	process	1	1	yes
Saskatchewan web, Jun-Dec 1995	method	12	1	no
Saskatchewan web, Jun-Dec 1995	status	9	1	no
Zookeeper, 27 days	level	3	0	no
Zookeeper, 27 days	thread	15	0	no

A field-level form of the rule, under which a field is watched only when every one of its values is seen every hour, watches 1 of 18 fields: nearly every real field has a rare value, a 500 status or a PUT, that is silent for hours in a normal week, and one such value fails the field. The rule of Section 2.6 watches each value on its own and admits 19 values; the detector on each of their series, at the default setting for comparison with the logs' own rows, gives:

Log	Field	Value	Rows a bucket	Months	False a month	Stops in 20 min	Stops in 1 h	Stops in 1 day	To a tenth in 1 h	To 30% in 1 day
ClarkNet web, 2 weeks 1995	method	`GET`	805	0.2	4.29	100%	100%	100%	100%	100%
ClarkNet web, 2 weeks 1995	method	`HEAD`	0	0.2	0.00	0%	0%	100%	0%	100%
ClarkNet web, 2 weeks 1995	status	`200`	711	0.2	4.29	100%	100%	100%	100%	100%
ClarkNet web, 2 weeks 1995	status	`304`	62	0.2	4.29	98%	100%	100%	100%	100%
ClarkNet web, 2 weeks 1995	status	`404`	16	0.2	4.29	73%	100%	100%	100%	100%
ClarkNet web, 2 weeks 1995	status	`302`	6	0.2	4.29	14%	100%	100%	96%	100%
Google cluster jobs, 29 days	event	`0`	74	0.7	0.00	100%	100%	100%	98%	100%
Google cluster jobs, 29 days	event	`1`	74	0.7	0.00	100%	100%	100%	98%	100%
Google cluster jobs, 29 days	event	`4`	44	0.7	0.00	100%	100%	100%	100%	100%
Google cluster jobs, 29 days	event	`5`	28	0.7	1.36	73%	100%	100%	88%	98%
OpenSSH, 28 days	kind	`Failed password`	0	0.7	8.74	1%	10%	100%	3%	70%
OpenSSH, 28 days	kind	`Received disconnect`	0	0.7	7.28	1%	6%	100%	3%	97%
OpenSSH, 28 days	kind	`pam_unix( sshd:auth): authentication`	0	0.7	4.37	1%	11%	100%	2%	69%
OpenSSH, 28 days	kind	`pam_unix( sshd:auth): check`	0	0.7	5.81	0%	8%	100%	2%	92%
OpenSSH, 28 days	kind	`Invalid user`	0	0.7	5.80	1%	4%	100%	2%	87%
OpenSSH, 28 days	kind	`input_ userauth_ request: invalid`	0	0.7	5.80	1%	4%	100%	2%	87%
OpenSSH, 28 days	process	`sshd`	0	0.7	2.91	1%	30%	100%	3%	71%
Saskatchewan web, Jun-Dec 1995	method	`GET`	33	6.2	2.10	83%	100%	100%	86%	95%
Saskatchewan web, Jun-Dec 1995	status	`200`	30	6.2	2.26	82%	100%	100%	88%	97%

On the values the per-value form watches the detector behaves as it does on a source. A value with rows in every bucket, GET, 200, the Google event types, is caught within the hour on every stop at the log's own false-alarm rate. A value with an empty median bucket, ClarkNet's HEAD and every OpenSSH kind, is judged at the long widths, caught within the day, and on OpenSSH pays 2.91 to 8.74 false alarms a month. The hour test is right about what a sender is, and the unit it applies to has to be the value: a field is almost never a sender, because nearly every field has a rare value. A value the test admits that still has an empty median bucket, as OpenSSH's kinds, is judged at the long widths like a thin source and pays the thin source's false alarms; a value the test refuses is not watched, and its stop goes unseen, which is the rule's cost. The quiet setting of the previous table applies to these values as to every sender.

4.6 Each width's share

Each width alone, the two short ones, and all four, on the real and modeled count series:

Widths	Series judged, real / modeled	False a month, real / modeled	Stops in 20 min	Stops in 1 h	Stops in 1 day	To a tenth in 1 h	To 30% in 1 h	To 30% in 1 day	Halving in 1 day
5 minutes alone	10 of 14 / 18 of 21	0.00 / 0.00	93% / 100%	94% / 100%	98% / 100%	49% / 82%	8% / 51%	31% / 84%	8% / 53%
30 minutes alone	13 of 14 / 21 of 21	0.19 / 0.20	1% / 0%	86% / 99%	98% / 100%	61% / 85%	19% / 56%	58% / 83%	33% / 68%
6 hours alone	14 of 14 / 21 of 21	0.00 / 0.10	0% / 0%	0% / 0%	100% / 100%	0% / 0%	0% / 0%	65% / 91%	41% / 82%
1 day alone	14 of 14 / 21 of 21	0.00 / 0.00	0% / 0%	0% / 0%	99% / 100%	0% / 0%	0% / 0%	59% / 89%	36% / 81%
5 and 30 minutes	13 of 14 / 21 of 21	0.19 / 0.20	62% / 86%	91% / 99%	98% / 100%	68% / 86%	21% / 56%	61% / 83%	36% / 68%
All four	14 of 14 / 21 of 21	0.16 / 0.31	57% / 86%	79% / 95%	100% / 100%	63% / 82%	19% / 56%	74% / 91%	52% / 88%

Each width is judged on the series and the buckets that fill it, and a share is over the drops injected at the buckets that row judges, so the rows have different denominators and do not add. "5 and 30 minutes" is scored on the 13 series and the hours those widths judge; "All four" also on the thin series' nights and on the series only the long widths judge, where no stop is caught within the hour, which is why its stops within the hour read 12 points lower than the two short widths' and its stops within the day read higher. On the buckets both rows judge the four widths catch what the two do: the long widths add readings, and a reading can only add an alarm. The false-alarm rates are likewise per month that row judges. The 5-minute width alone is judged on 10 of the 14 real series, raises no false alarm, and is the stop's width: 93% of stops within 20 minutes on the series it judges, and 31% of drops to 30% within the day. The 30-minute width alone carries the real false alarms, 0.19 a month, and most of the degradation caught within the hour. The 6-hour and 1-day widths alone catch no stop within the hour and nearly every one within the day, and 59% to 65% of drops to 30% within the day on real series; they are the day's widths. The four together take each width's catch at its own horizon, the 5-minute speed on a stop, the 30-minute rate on a drop to a tenth, the day's drops at the long widths, for 0.16 / 0.31 false alarms a month: on the modeled series the long widths add 0.10 a month to the 30-minute width's, and on the real series they add no alarm, the rate reading lower than the 30-minute width's alone because it is spread over the fourteenth series' months as well.

4.7 The gap

The gap scored alone, on every source with ten completed spells in its history: the six thin tweet series, the thin daily shape, the sparse sources and eight of the ten logs. A dense source has no empty bucket and is not judged. "Longest normal spell" is the longest learned spell, averaged over the judged time; a stop is confirmed when the current spell is m times that, and the median is the time from the stop to that confirmation. The count widths' numbers on the same sources are in Sections 4.1 to 4.4, and Section 4.8 puts the two coordinates together.

Source	Spells judged on	Longest normal spell	Margin 1: false a month	Margin 1.5: false a month	stop caught within	median	Margin 2: false a month	median
Thin tweet series (CRM, IBM, KO, UPS, PFE, CVS)	842	2.1 h	0.19	0.00	82% in 6 h, 99% in 1 day	4 h	0.00	5 h
Thin daily (modeled)	751	2.2 h	2.86	0.00	100% in 6 h, 100% in 1 day	3 h	0.00	4 h
12 rows an hour	1,884	0.6 h	2.14	0.00	100% in 6 h, 100% in 1 day	60 min	0.00	80 min
4 rows an hour	1,634	1.8 h	2.14	0.00	100% in 6 h, 100% in 1 day	3 h	0.00	3 h
1 row an hour	604	7.3 h	1.88	0.00	1% in 6 h, 100% in 1 day	11 h	0.00	15 h
1 row a day	25	90.4 h	2.41	0.00	0% in 6 h, 0% in 1 day	5 days	0.00	7 days
Apache error log, 264 days	539	6.2 h	10.58	1.91	13% in 6 h, 100% in 1 day	9 h	1.91	12 h
BGL supercomputer, 215 days	883	11.3 h	8.19	0.96	2% in 6 h, 100% in 1 day	16 h	0.16	26 h
Calgary web, 1994-95	1,079	1.9 h	3.98	0.83	91% in 6 h, 100% in 1 day	3 h	0.56	5 h
Linux syslog, 264 days	113	23.8 h	7.88	1.02	1% in 6 h, 22% in 1 day	31 h	0.13	43 h
NASA web, Jul-Aug 1995	24	0.7 h	0.00	0.00	100% in 6 h, 100% in 1 day	65 min	0.00	85 min
OpenSSH, 28 days	772	1.1 h	47.58	5.60	100% in 6 h, 100% in 1 day	2 h	2.80	4 h
Saskatchewan web, Jun-Dec 1995	22	0.2 h	1.17	0.50	100% in 6 h, 100% in 1 day	20 min	0.17	25 min
Zookeeper, 27 days	43	11.5 h	9.12	9.12	23% in 6 h, 100% in 1 day	12 h	7.60	17 h

At margin 1 the gap alarms about as often as the bound says a healthy source will beat its own record: 2.14 a month pooled over the sparse sources, 2.86 on the thin daily shape, 0.19 on the thin tweets. Those are the sources that do nothing wrong, and a healthy source sets a new longest silence about once per history length, so margin 1 is about a false alarm a month by construction, and more where a spell that alarmed is not learned and the record it beat stands to be beaten again. At margin 1.5 the gap raises no false alarm on any count series, modeled shape or sparse source, and 51 alarms in 41.0 source-months on the eight logs, 1.24 a month pooled, from none on NASA to 9.12 on Zookeeper, a 27-day log judged from its second week on 43 spells whose longest runs 11.5 hours; at margin 2 the logs raise 31. The gap is faster than the count widths on every sparse source: a stop is confirmed at a median of 60 minutes at 12 rows an hour where the 6-hour width took 3 hours, 3 hours at 4 rows an hour against 4 hours, 11 hours at a row an hour against 15 hours, and 5 days at a row a day, which no count

width judges. On the thin tweet series it confirms a stop at a median of 80 minutes to 9 hours, within the day on all of them. The one log on which it is slow is the Linux syslog, whose normal silences run 23.8 hours, so a stop is confirmed after 31 hours: the gap cannot be faster than the source's own habit, and it says so by the number it reports.

4.8 Count widths and gap together

This is the detector as an operator sees it: five widths on every source, each judged where its gate admits it, and a stop caught by whichever fires first (Section 2.8). The count widths and the gap are scored together on every count series, modeled series, sparse source and log: a bucket is judged when either coordinate judges it; a false alarm is a start of either's alarm, the two counted once where they overlap; a stop injected at a judged bucket is caught at the earlier of the two delays, the count widths' within 25 hours and the gap's at any delay. Rates are means over the sources of a row, as everywhere. Every share is over the buckets either coordinate judges, so the count widths' own columns here read lower than in Sections 4.1 to 4.4, where their shares are over the buckets they judge alone. The eight count series the gap never judges, having no ten completed spells on them, are one row; Tweets KO, dense by its median but with a quiet night, is judged in both coordinates and has its own row. The rates here differ from Sections 4.1, 4.2 and 4.7 because the months differ: a source's months are the buckets either coordinate judges.

Source	Months judged by either	False alarms a month, count widths / gap / together	Stops within the hour, count widths / gap / together	Stops within the day, count widths / gap / together	Stops caught at all, together	Median stop, together
8 count series the gap never judges	1.2	0.14 / 0.00 / 0.14	99% / 0% / 99%	100% / 0% / 100%	100%	20 min
Tweets KO (8 a bucket)	0.8	0.00 / 0.00 / 0.00	100% / 44% / 100%	100% / 99% / 100%	100%	20 min
Tweets IBM (3 a bucket)	0.8	1.19 / 0.00 / 1.19	88% / 1% / 89%	100% / 99% / 100%	100%	45 min
Tweets CRM (2 a bucket)	0.9	0.00 / 0.00 / 0.00	72% / 1% / 72%	100% / 100% / 100%	100%	45 min
Tweets UPS (2 a bucket)	0.9	0.00 / 0.00 / 0.00	54% / 0% / 54%	100% / 99% / 100%	100%	45 min
Tweets CVS (0 a bucket)	0.8	0.00 / 0.00 / 0.00	0% / 0% / 0%	100% / 100% / 100%	100%	6 h
Tweets PFE (0 a bucket)	0.9	0.00 / 0.00 / 0.00	2% / 0% / 2%	100% / 99% / 100%	100%	3 h
Modeled: steady	0.9	0.00 / 0.00 / 0.00	100% / 0% / 100%	100% / 0% / 100%	100%	20 min
Modeled: steady, falling 3%/day	0.9	0.00 / 0.00 / 0.00	100% / 0% / 100%	100% / 0% / 100%	100%	20 min
Modeled: variable, falling 3%/day	0.9	1.07 / 0.00 / 1.07	100% / 0% / 100%	100% / 0% / 100%	100%	20 min
Modeled: daily	0.9	0.00 / 0.00 / 0.00	100% / 0% / 100%	100% / 0% / 100%	100%	20 min
Modeled: variable	0.9	1.07 / 0.00 / 1.07	100% / 0% / 100%	100% / 0% / 100%	100%	20 min
Modeled: weekday/weekend	0.9	0.00 / 0.00 / 0.00	100% / 0% / 100%	100% / 0% / 100%	100%	20 min
Modeled: thin daily	0.9	0.00 / 0.00 / 0.00	65% / 0% / 66%	100% / 100% / 100%	100%	45 min
Sparse: 12 rows an hour	1.9	0.00 / 0.00 / 0.00	0% / 54% / 54%	100% / 100% / 100%	100%	60 min
Sparse: 4 rows an hour	1.9	0.00 / 0.00 / 0.00	0% / 0% / 0%	100% / 100% / 100%	100%	3 h

Source	Months judged by either	False alarms a month, count widths / gap / together	Stops within the hour, count widths / gap / together	Stops within the day, count widths / gap / together	Stops caught at all, together	Median stop, together
Sparse: 1 row an hour	1.9	0.00 / 0.00 / 0.00	0% / 0% / 0%	100% / 100% / 100%	100%	10 h
Sparse: 1 row a day	1.9	0.00 / 0.00 / 0.00	0% / 0% / 0%	0% / 0% / 0%	100%	5.0 d
ClarkNet web, 2 weeks 1995	0.2	4.29 / 0.00 / 4.29	100% / 0% / 100%	100% / 0% / 100%	100%	20 min
Google cluster jobs, 29 days	0.7	0.00 / 0.00 / 0.00	100% / 0% / 100%	100% / 0% / 100%	100%	20 min
NASA web, Jul-Aug 1995	1.0	1.02 / 0.00 / 1.02	89% / 0% / 89%	100% / 100% / 100%	100%	20 min
Saskatchewan web, Jun-Dec 1995	6.2	2.26 / 0.48 / 2.59	100% / 91% / 100%	100% / 97% / 100%	100%	20 min
Calgary web, 1994-95	10.8	6.39 / 0.83 / 6.67	42% / 0% / 42%	100% / 100% / 100%	100%	95 min
OpenSSH, 28 days	0.7	2.80 / 5.60 / 4.20	28% / 25% / 34%	100% / 100% / 100%	100%	95 min
Apache error log, 264 days	7.8	5.36 / 1.91 / 3.95	1% / 0% / 1%	66% / 100% / 100%	100%	8 h
BGL supercomputer, 215 days	6.2	17.03 / 0.96 / 16.87	3% / 0% / 3%	100% / 100% / 100%	100%	6 h
Linux syslog, 264 days	7.9	5.34 / 1.02 / 5.59	1% / 0% / 1%	79% / 20% / 85%	100%	19 h
Zookeeper, 27 days	0.7	0.00 / 9.12 / 9.12	0% / 5% / 5%	0% / 100% / 100%	100%	10 h

Every injected stop on every one of the 35 sources is caught, and the two coordinates divide the sources between them. Within the day the detector catches 100% of stops on the real count series, 100% on the modeled series, 75% on the sparse sources and every stop on every log but the Linux syslog, whose normal silences run a day, so that a stop there is confirmed at a median of 19 hours and 85% within the day; the other exception to the day is the source with a row a day, whose stop is confirmed at a median of five days, the gap's margin over its longest normal silence. On the eight dense count series and the dense modeled shapes the count widths catch a stop at a median of 20 minutes, 99% within the hour on the real eight, and the gap is not judged; on Tweets KO both coordinates judge, and the count widths are the faster by far. On the sparse sources, on Zookeeper and on the Apache error log the gap is what catches the stop within the day, and at a dozen rows an hour it catches half the stops within the hour where the 6-hour width catches none. On the thin tweet series the 30-minute width is the faster where it judges and the gap holds the night.

The false alarms of the five widths together are the count widths' with the gap's added where the gap judges. The five raise 0.16 false alarms a month on the real count series, the count widths' 0.16 and nothing from the gap, and 5.43 a month over the logs against the count widths' 4.45. The difference is the gap's alarms on Zookeeper and OpenSSH, the two logs under a month long that it judges from their second week on a short record. On the Apache error log the five together raise fewer than the count widths alone, because a silence the gap alarms on bridges two count alarms that were counted twice into one.

5 The trade

Section 4 said what is caught. This section says what it costs, in the order of the three needs: stops cost time and not alarms (5.1); degradations trade against false alarms, and the margin

and the wait set the rate of exchange (5.2); the baseline's detection was never usable at its alarm rate (5.3); and each part of the detector has a measured reason to be there (5.4).

5.1 Full stops cost time, not alarms

Every setting in Sections 2.7 and 5.2 catches the same stops on the same series: 100%, 100% and 100% of real stops within the day at the fast, default and quiet settings, and 80%, 79% and 79% within the hour. A stop is a reading of zero, and zero is below the band wherever the lower edge is positive. The edge $\ell_w(t)$ is positive exactly when $q_w\hat{}–(\mathrm{t}) > 1/(M_w(t) + 1)$. A ratio $(S_w(u) + 1)/(M_w(u) + 1)$ of an empty window is $1/(M_w(u) + 1)$, which is at most $1/(M_w(t) + 1)$ when the window's time of day is at least as busy as now, so the edge is zero when the order statistics the quantile interpolates come from empty windows at times of day at least as busy as now: at the short widths, whose edge sits between the second- and third-lowest of about 2,000 ratios, two or more such windows in the history, at the long widths, between the fifth- and sixth-lowest of about 8,000, five or more. On the count series the band judges that is rare, and every real stop is caught within the day (Section 4.1); on a bursty log it is common, which is why the count widths catch 66% of the Apache error log's stops and the gap the rest (Section 4.8). The gate does not make the edge positive; it removes the widths whose windows usually hold too few rows for their ratios to mean anything. Where the edge is positive, a stop is below the band at the first reading that no longer holds the rows before the stop. What the setting changes is the wait, k readings, and what the source changes is the width: the stop is caught about k readings after the narrowest judged window has emptied, 20 minutes on a dense source at $k = 4$, 45 minutes where only the 30-minute width is judged, hours at the long widths. The operator picking a setting is not trading stops away. The operator picking a source's sampling, how many rows it emits, is choosing how soon its stop can be seen.

5.2 Degradations trade against false alarms

The margin and the wait are the two knobs. At the design's widths, history and gate:

δ	k	False a month, real / modeled	Stops in 1 h	To a tenth in 1 h	To 30% in 1 h	To 30% in 1 day	Halving in 1 day	Cost at $K\lambda = 30$, real / modeled
0.2	3	0.74 / 0.51	80% / 95%	64% / 83%	23% / 58%	74% / 91%	54% / 88%	14.1 / 6.8
0.2	4	0.16 / 0.31	79% / 95%	63% / 82%	19% / 56%	74% / 91%	52% / 88%	14.1 / 7.0
0.3	3	0.48 / 0.15	80% / 95%	62% / 80%	18% / 52%	70% / 91%	46% / 82%	14.5 / 7.4
0.3	4	0.08 / 0.10	79% / 95%	60% / 79%	14% / 50%	69% / 91%	43% / 82%	14.7 / 7.7
0.4	3	0.32 / 0.00	80% / 95%	59% / 77%	11% / 45%	62% / 91%	34% / 66%	15.4 / 8.3
0.4	4	0.08 / 0.00	79% / 95%	57% / 76%	9% / 43%	62% / 91%	31% / 64%	15.6 / 8.7

The wait is the cheaper knob. From $k = 3$ to $k = 4$ at $\delta = 0.2$ the false alarms fall from 0.74 to 0.16 a month on real series for one point of drops to a tenth and four of drops to 30% within the hour, and 5 minutes on a stop. From $\delta = 0.2$ to 0.3 at $k = 4$ they fall from 0.16 to 0.08 for three and five points; $\delta = 0.4$ buys nothing more on the real series and costs three points more of drops to a tenth and five of drops to 30%. Stops within the hour do not move anywhere in the grid. The cost column at $K\lambda = 30$, computed from the unrounded shares, runs from 14.1 to 15.6 on real series: a difference of 0.5 in it is a point of the share missed or half an alarm

a month, and the rule below treats a difference under 0.5 as a tie and one of 0.5 or more as a ranking. The default sits where the false alarms stop falling fast.

An operator who knows how often a source really drops, λ a month, and what a missed drop costs in false alarms, K, picks the setting that minimizes $F + K\lambda M$, M the share missed. Only the product $K\lambda$ matters; the last column is that cost at $K\lambda = 30$, a drop a month with a miss worth 30 false alarms, or a drop a quarter worth 90, with the share missed taken as the mean over stops, drops to a tenth and drops to 30% within the hour, and "detected" below is one minus that share. The real series were split into two halves of seven by name and the setting with the least cost on one half was scored on the other (heldout.py): half A picks $\delta = 0.2$, $k = 4$, half B picks $\delta = 0.2$, $k = 3$, and the modeled series alone pick $\delta = 0.2$, $k = 3$, which on the real series scores 0.74 false alarms a month with 56% detected where $k = 4$ scores 0.16 and 54%. The split settles the margin, $\delta = 0.2$ on every half, and does not settle the wait: $k = 3$ and $k = 4$ are within three points of each other in detection on every half, tie on the cost column at 14.1, and differ in false alarms, 0.74 against 0.16 a month, about 9 alarms against 2 in 15 source-months. One rule picks the default: among the settings the cost column does not separate, take the quietest. That gives $k = 4$ over $k = 3$, and $\delta = 0.2$ over $\delta = 0.3$ because the cost column does separate them, 14.1 against 14.7, the quiet setting giving up three points of drops to a tenth and five of drops to 30% within the hour for 0.08 false alarms a month. The quiet setting is offered for an operator who weighs a false alarm more than the cost column's $K\lambda = 30$ does, and senders take it (Section 2.7); Section 5.3 shows a model under which that weighting is the right one.

The three settings of Section 2.7 are the frontier's useful stretch:

Setting	k	δ	False a month, real / modeled	Stops in 15 min	Stops in 20 min	Stops in 1 h	Stops in 1 day	To a tenth in 1 h	To 30% in 1 h	To 30% in 1 day	Halving in 1 day
Fast	3	0.2	0.74 / 0.51	58% / 86%	58% / 86%	80% / 95%	100% / 100%	64% / 83%	23% / 58%	74% / 91%	54% / 88%
Default	4	0.2	0.16 / 0.31	0% / 0%	57% / 86%	79% / 95%	100% / 100%	63% / 82%	19% / 56%	74% / 91%	52% / 88%
Quiet	4	0.3	0.08 / 0.10	0% / 0%	57% / 86%	79% / 95%	100% / 100%	60% / 79%	14% / 50%	69% / 91%	43% / 82%

5.3 Why the baseline's detection is on paper only

The baseline's 84% of drops to a tenth rests on 145 false alarms per source-month, about five a day per source. An operator who acts on every alarm acts on a false one 99.4% of the time when a source drops once a month; one who acts on none misses every drop. Model the operator as acting on an alarm with probability equal to its precision, $\lambda R/(\lambda R + F)$, where λ is the rate of real drops, R the share the rule detects and F its false-alarm rate: then the share of drops acted on is R times that precision. For a source that drops to a tenth once a month, the baseline's precision is 0.6% and it gets 0.5% of the drops acted on; the recommended detector's precision is 79% and it gets 50% (extend_grid2.json, Section 9). Under this model, on drops to a tenth once a month, the quiet setting is best: $\delta = 0.3$, $k = 4$ gets 0.53 of the drops acted on, against 0.50 for the default and 0.49 for $\delta = 0.4$, $k = 4$, which misses more than its quiet buys. The two models disagree at the margin, the cost column of Section 5.2 by 0.6 for the default, this one by three points for quiet, because this one prices a false alarm by the operator's fatigue and scores one drop size where the cost column weighs three. The default follows the cost column; the quiet setting is the row for an operator whose alarms are read by people, and it is the setting for senders, whose false alarms add over a hundred of them (Section 2.7). The replay cannot supply λ, nor the cost of a missed drop; what it can say is that at the baseline's rate every detected

drop is one alarm among a hundred and fifty.

5.4 What each part contributes

The detector is the two short widths of an earlier design plus three parts: the long widths, the 28 days of history behind them, and the gate with the typical window it tests. Each was added for a measured reason, and each is removable. Each row judges its own bucket set and its shares are over that set: the rows without the gate judge the thin series at the short widths too, so their shares include buckets the gated rows do not.

Detector	False a month, real / modeled	Alarms on the real series	Stops in 20 min	Stops in 1 h	Stops in 1 day	To 30% in 6 h	To 30% in 1 day	Halving in 6 h	Halving in 1 day
Four widths, 7 days behind every width, 10-row gate	5.05 / 0.56	49	57% / 86%	80% / 95%	100% / 100%	69% / 87%	94% / 91%	39% / 67%	75% / 90%
5 and 30 minutes, 7-day history, no gate	0.25 / 0.26	3	59% / 87%	80% / 97%	92% / 100%	39% / 75%	55% / 86%	17% / 53%	35% / 71%
Four widths, 28 days behind the long ones, no gate	0.25 / 0.36	3	59% / 87%	80% / 97%	100% / 100%	53% / 85%	74% / 91%	27% / 62%	52% / 88%
Four widths, 28 days behind the long ones, 10-row gate, typical level summed from buckets	0.17 / 0.26	2	57% / 86%	78% / 95%	96% / 100%	56% / 84%	72% / 91%	31% / 62%	51% / 88%
Four widths, 28 days behind the long ones, 10-row gate, typical window (recommended)	0.16 / 0.31	2	57% / 86%	79% / 95%	100% / 100%	52% / 85%	74% / 91%	27% / 62%	52% / 88%

The long widths cost nothing within the hour and buy the day: drops to 30% caught within the day go from 55% to 74% on real series and halvings within the day from 35% to 52%. They need their 28 days because a width's quantile is only as good as the number of windows that do not overlap behind it: a week holds 2,000 half-hour windows and 7 days, and a 1-day width learned from 7 days alarms on the first day that is 20% under the worst of the week. With 7 days behind every width the real series raise 49 alarms, 5.05 a month, against 2 with 28 days behind the long ones (first row; Section 6.5 adds each long width in turn to the two short ones with 7 days). The gate removes a width where its reading cannot mean anything, a window that usually holds a handful of rows, and this is where the thin series' false alarms were: 3 alarms on the real series without it, 2 with it. The typical window it tests must be the median of the same window on past days, not a sum of per-bucket medians: on the Apache error log, two hundred lines a day in a tenth of its buckets, the per-bucket median is zero at every time of day, the summed level is zero at every width, and the gate refuses the source. Section 4.2's thin logs are judged under the window definition and were not under the summed one.

The gate's bound is a constant, not a knob. Its value is set by what a quantile of ratios can mean with few rows: a window that holds a handful of rows takes a handful of distinct ratio

values, and the lowest 0.05% of them is noise. Sweeping it at the design:

Gate, rows	False a month, real / modeled	Alarms on the real series	Real months judged	Stops in 20 min	Stops in 1 h	Stops in 6 h	Stops in 1 day	To a tenth in 1 h	To 30% in 1 day
5	0.25 / 0.31	3	14.8	59% / 87%	80% / 97%	94% / 100%	100% / 100%	63% / 82%	74% / 91%
10	0.16 / 0.31	2	14.8	57% / 86%	79% / 95%	94% / 100%	100% / 100%	63% / 82%	74% / 91%
20	0.08 / 0.31	1	14.8	46% / 86%	72% / 92%	93% / 100%	100% / 100%	62% / 80%	74% / 91%
40	0.08 / 0.31	1	14.8	35% / 85%	63% / 86%	89% / 99%	100% / 100%	59% / 74%	74% / 91%

Ten rows is the smallest gate that removes the alarm a handful of rows can produce without moving the stops, and the modeled series, which fill every width, barely see the gate. At 5 rows the 30-minute width judges the series at 2 to 3 rows a bucket: one of their alarms returns, and their stops are caught at that width, which lifts stops within 20 minutes on real series from 57% to 59%. At 20 rows that width is gated out on those series, one more of their alarms goes, their stops wait for the 6-hour width, and stops within 20 minutes fall to 46%; at 40 rows no further alarm goes and they fall to 35%.

6 Alternatives tried

Every choice in Section 2 was made against an alternative that was replayed and lost, and this section keeps the record. The early harness (`normal_lab.py`, Section 9) keeps the baseline's judging and learning rules: windows under an alarm on either side are not learned and the band is refreshed every reading.

Section	Question	Harness
6.1	Which band: the twelve candidates that led to pooled ratios with a margin and a wait	early
6.2	The margin and the wait at one width	early
6.3	Speed: a CUSUM, and widths between 5 and 30 minutes	early
6.4	What is learned under an alarm	the detector as written
6.5	Which long width, and with how much history	the detector as written
6.6	Widths defined in rows rather than minutes	the detector as written
6.7	A typical level per group of weekdays	the detector as written
6.8	The alternatives to the gap and to a value's own series	two of their own, on the modeled sender mixes

6.1 Pooling, margin and wait

Twelve bands at the 30-minute width, each keeping the baseline's judging and learning rules except that all but the first exclude an alarm's first window from learning.

Candidate	Band	δ	k	Windows it rests on
Baseline	99% of the same time of day's windows	0	1	about 91
Baseline, first window not learned	as baseline	0	1	about 91
Baseline + 10%	Baseline band, then 10% wider each side	0.1	1	about 91
Pooled raw + 10%	99.9% of every window of the week, raw	0.1	1	about 2,000
Ratios	$m(t) \times$ 99.9% band of every window's ratio to its own time of day's typical level	0	1	about 2,000
Ratios + 10%	Ratios, then 10% wider each side	0.1	1	about 2,000
Negative-binomial + 10%	$m(t) \pm$ 99.9% band of every window's residual, scaled by a fitted variance	0.1	1	about 2,000
Ratios + 10%, 3-day half-life	Ratios + 10%, each window weighted by 0.5^(age in days / 3)	0.1	1	about 2,000, fewer in effect
Ratios + 10%, day groups	Ratios + 10%, with the typical level learned per group of weekdays found by clustering	0.1	1	about 2,000
Ratios + 20%	Ratios, then 20% wider each side	0.2	1	about 2,000
Ratios + 10%, 3 in a row	Ratios + 10%, alarmed only on the third consecutive reading below; those readings not learned	0.1	3	about 2,000
Ratios + 20%, 3 in a row	Ratios + 20%, alarmed only on the third consecutive reading below	0.2	3	about 2,000

Day groups follow the shape of Splunk's assisted thresholding (Rajyaguru, Bai and Curtis, 2023), which clusters one-day subsequences and keeps the clustering only when the median silhouette passes a cutoff. The choices here are ours: each of the last 28 days becomes 24 hourly sums, days are clustered into two or three groups, the grouping is kept if its median silhouette is at least 0.2, and each weekday joins the group most of its days fell in.

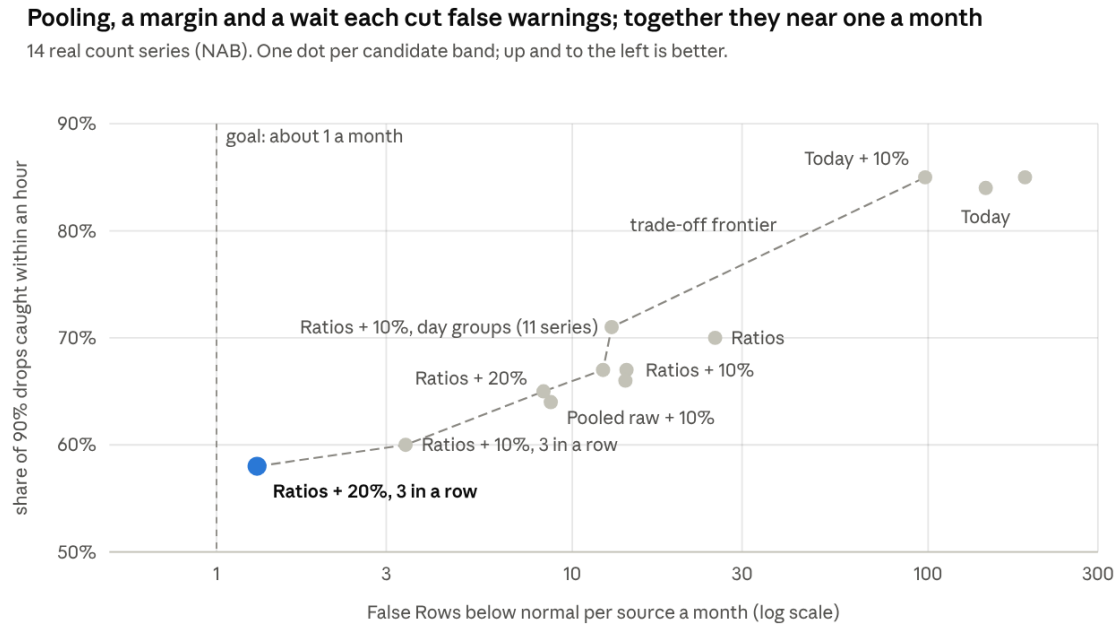


Figure 1. False alarms per source-month against the share of drops to a tenth caught within the hour, for twelve bands at the 30-minute width; 14 real series, means per series. The dashed line joins the settings no other setting beats on both axes.

False alarms per source-month, by series:

Candidate	Steady	Variable	Daily	Thin daily	Weekday/ weekend	Steady, falling 3%/day	Variable, falling 3%/ day	Tweets (10)	NYC taxi	AWS (3)
Baseline	146.8	137.2	136.7	102.5	88.8	491.6	305.6	132.3	93.5	205.8
Baseline, first window not learned	198.2	184.2	164.1	185.3	103.2	493.0	352.5	174.4	120.0	252.0
Baseline + 10%	10.7	98.6	58.9	91.4	77.5	198.3	204.1	118.1	7.1	61.6
Pooled raw + 10%	0.0	7.5	1.8	0.0	7.1	5.4	15.0	10.6	1.3	4.8
Ratios	31.8	20.4	18.2	10.0	11.8	349.5	108.3	29.5	14.7	14.4
Ratios + 10%	0.4	8.2	5.7	10.4	4.6	12.9	19.6	16.8	1.6	9.6
Negative-binomial + 10%	0.4	8.2	6.1	9.6	4.3	12.1	19.6	16.7	1.6	9.6
Ratios + 10%, 3-day half-life	0.4	7.9	5.0	8.9	4.3	7.9	13.6	14.1	1.1	9.6
Ratios + 10%, day groups	0.4	7.5	2.5	13.6	6.1	12.9	20.7	13.8	4.0	n/a (14 days)
Ratios + 20%	0.0	4.6	0.7	8.2	0.0	0.0	3.6	9.7	0.0	6.4
Ratios + 10%, 3 in a row	0.4	5.7	2.1	0.7	1.8	1.4	11.4	4.2	1.5	1.6
Ratios + 20%, 3 in a row	0.0	2.5	0.0	0.7	0.0	0.0	1.4	1.9	0.0	0.0

Drops detected within an hour, as a share of judged buckets:

Candidate	Real: stops	Real: halvings	Modeled: stops	Modeled: to a tenth	Modeled: halvings
Baseline	86%	59%	98%	97%	79%
Baseline, first window not learned	87%	64%	98%	97%	81%
Baseline + 10%	87%	57%	98%	97%	75%
Pooled raw + 10%	78%	15%	86%	75%	42%
Ratios	78%	23%	98%	93%	59%
Ratios + 10%	79%	15%	98%	91%	49%
Negative-binomial + 10%	84%	14%	97%	90%	50%
Ratios + 10%, 3-day half-life	78%	14%	98%	91%	48%
Ratios + 10%, day groups	81%	22%	98%	97%	52%
Ratios + 20%	79%	10%	98%	88%	40%
Ratios + 10%, 3 in a row	76%	9%	97%	88%	45%
Ratios + 20%, 3 in a row	76%	6%	97%	85%	37%

Read down the rows on the real series. The baseline raises 145 false alarms a month, 89 to 492 by series, more than its 91 windows suggest, because the windows are dependent and the edge flickers. Leaving an alarm's first window out of its learning raises that to 187, 103 to 493 by series, since the baseline leans on those windows to stay wide. Widening it 10% brings the steady series to 11 and the taxi series to 7 and leaves the rest at 59 to 204. Pooling the week's raw windows is as quiet as ratios but blind at quiet hours: no stop is detected on the thin daily series and 42% of drops to a tenth on the weekday/weekend series. Ratios take 145 false alarms a month to 25. A 10% margin on them takes 25 to 14. Negative-binomial residuals match ratios within one alarm a month, and add a fitted variance. A three-day half-life trims false alarms by 14 to 23%, and adds a fitted weight. Day groups lift drops to a tenth on the weekday/weekend series from 63% to 100%, and on trending series the clustering sorts days by age, not weekday. A 20% margin takes 25 to 8. Three consecutive readings take 8 to 1.3, 0.7 on the modeled series.

The baseline's false alarms are edge flicker: the window sits a median of 5% under the lower edge, the alarm lasts a median of 20 minutes, half come within two hours of the previous one. With ratios + 20% and four readings (patterns.py), the real series raise 8 alarms in about 15 source-months, all under 30 minutes, a median of 31% under the edge, half on one tweet series in its morning hours.

6.2 The margin and the wait at one width

Twenty settings of the ratio band at the 30-minute width: margins $\delta \in \{0, 0.1, 0.2, 0.3, 0.4\}$ and waits $k \in \{1, 2, 3, 4\}$. On real series false alarms range 150-fold, from 0.17 to 25 a month, while drops detected move from 45% to 64%.

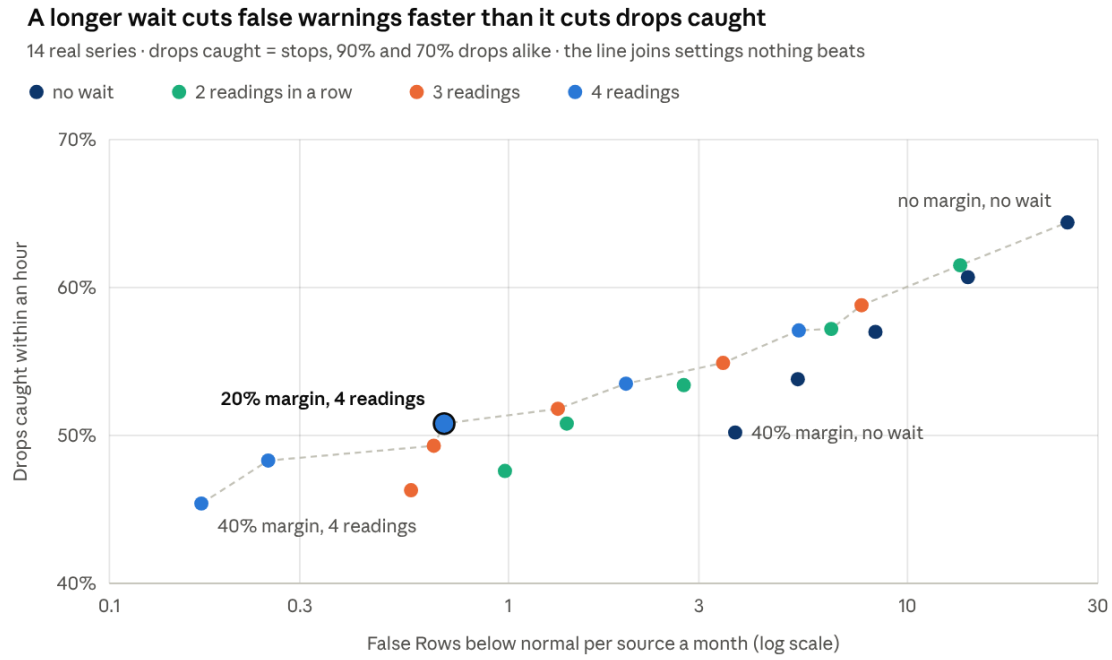


Figure 2. The false-alarm/missed-drop frontier over margins 0 to 40% and waits 1 to 4 readings, ratio band at the 30-minute width; 14 real series, means per series.

Stops barely trade: every setting detects 76% to 79% of stops on real series, and the stops missed are on series too sparse for a 30-minute window. The trade is in partial drops: from $\delta = 0.2$, $k = 4$ to $\delta = 0$, $k = 1$ multiplies false alarms 36-fold and lifts drops to 30% detected from 20% to 46%. Waiting is cheaper than widening: $\delta = 0.2$, $k = 4$ raises 0.69 a month and detects 51% of drops; $\delta = 0.3$, $k = 3$ raises 0.65 and detects 49%; every setting on the frontier under 5 false alarms a month has $k \geq 3$.

6.3 Speed: a shorter width against a CUSUM

A stop must be detected fast. The baseline detects a stop after a median of 15 minutes on real series (90% within 30); the 30-minute width with a margin and a wait takes a median of 40 minutes (90% within 45): about 25 minutes for the window to empty past the wider band, then the readings waited.

Two routes to speed were replayed. The first, a rank-based CUSUM, cannot be both fast and quiet. Each 5-minute bucket scores $-\log_{10}$ of the rank of its ratio among the past week's pooled ratios (Section 2.2), a floor κ is subtracted, the excess accumulates over a run of low buckets, and the run alarms at a bar H (Page, 1954, with ranks for residuals). A bucket can be no rarer than one in a week, about 3.3 decades, so the evidence one reading adds is bounded by $3.3 - \kappa$, and the (κ, H) that is quiet on the modeled series needs eleven empty readings to alarm. At that setting it raised 1.06 false alarms a month averaged over the real series, 5.5 pooled, almost all on the taxi series, where correlated dips accumulate, and detected 54% of stops within the hour. A CUSUM on standardized residuals was noisier still: tweets and taxi rides have heavier tails than a Gaussian, and a dip lasts about 50 minutes, so its buckets are not independent evidence and one low reading is followed by several more.

The second route is the band itself at a shorter width, which is the design. The 5-minute width is quiet because its reading is below its band only when the count is near nothing, and fast because the k-th empty bucket comes at 5k minutes. Widths between 5 and 30 were replayed

on the early harness (`unified_lab.py`): at $\delta = 0.3$, $k = 3$, adding 15 minutes, or 10 and 15, to the two widths raised 1.75 and 2.00 false alarms a month on the real series, means over series, against 1.16 for the two alone under that harness, and caught the same stops, 80% within the hour at all three.

6.4 Learning

On the ratio band at the 30-minute width at $\delta = 0.2$, $k = 4$, on modeled step series that double, halve or stop on day 35; what is learned does not depend on the margin. Learning the windows under an above-normal alarm cuts above-normal alarms on real series from 3.4 to 1.3 per source-month, and a steady source that doubles is learned within an hour instead of 9 to 56 hours; on the daily shape a doubling takes 14 to 32 hours either way on two seeds and on the third is learned only as the replay ends, after 502 hours, and nothing below changes. Learning the windows under a below-normal alarm at a reduced weight fails: the band's edge is the rarest of 2,000 windows, so even a small weight on the new level moves it, and a source that halves is learned within 3 hours on the steady shape and 10 on the daily at weight 0.1 or 0.01, where without the weight the alarm stands up to 13 hours, until the readings that never reached k have taught the new level; a stopped source's alarm ends a day sooner at either weight, 96 hours against 121, because with no usable band the alarm can only end on a reading inside the band. An alarm at one width unlearns the buckets of its own readings only, w per reading, so a 5-minute alarm takes one bucket a reading out of learning, and with it the long windows that overlap that bucket.

6.5 Which long width, and with how much history

A third width was added to the two short ones at $\delta = 0.3$, $k = 3$ with 7 days of history, scored within the hour and at 6 hours and 1 day; every cell is real series / modeled series.

Third width	False a month	To 30%, within 1 h	within 6 h	within 24 h	Halving, within 1 h	within 6 h	within 24 h
none	0.49 / 0.20	21% / 53%	36% / 73%	52% / 85%	5% / 26%	15% / 46%	33% / 65%
1 hour	1.61 / 0.20	22% / 54%	47% / 79%	68% / 90%	5% / 26%	19% / 51%	39% / 73%
3 hours	1.41 / 0.20	21% / 53%	58% / 84%	77% / 91%	5% / 26%	30% / 56%	51% / 80%
6 hours	0.74 / 0.20	21% / 53%	58% / 85%	78% / 91%	5% / 26%	30% / 58%	52% / 85%
1 day	1.41 / 0.20	20% / 53%	35% / 73%	80% / 91%	4% / 26%	14% / 46%	53% / 85%

Within the hour nothing changes at any width: a halving does not pull a long sum below its band in an hour. Each width pays at its own horizon, and with 7 days behind it each pays in false alarms on the thin series, where a long window's lowest ratio of the week sits near zero: the 1-hour and 3-hour widths added 14 and 11 alarms on the thin tweet series, the 1-day width 5, two on the thin tweet series and three on the EC2 series whose bursts set its level. The 6-hour width was the cheapest, 3 alarms, and bought the most within its horizon. The design keeps 6 hours and 1 day and gives both 28 days of history, which is what removes those alarms (Section 5.4).

6.6 Widths defined in rows

Instead of fixed widths with a gate, the widths were defined from the source's rate: a ladder of rungs from 5 minutes to a day, the fast width the shortest rung whose typical window holds B rows, the slow width the shortest holding 6B, picked afresh at each reading, with 7 days of history behind every rung. At B = 10 it raised 26 false alarms on the real series against 6 for

the two fixed short widths at $\delta = 0.3$, $k = 3$, the setting of this experiment, all 26 on four of the thin tweet series, where a rung of hours or a day was chosen and its 7-day quantile was the worst of seven days. Holding the two short widths as a floor and lengthening only from there changed nothing, 26 alarms in the same places. At B = 20 the alarms fell to 5 but the thin series' stops caught within the hour fell with them, to 56% on IBM, 48% on CRM and 18% on UPS, since the fast width had moved to an hour or more. The ladder's failure was the history, not the idea: fixed widths with the gate and 28 days behind the long ones (Section 5.4) judge a thin source at the widths it fills, which is what the ladder was for, with one rule instead of a chooser.

6.7 Day groups

The one modeled series on which the detector misses drops to a tenth is the weekday/weekend shape, because a weekend's windows sit at a fifth of a weekday's inside the same pooled ratios. The remedy of Section 6.1, a typical level learned per group of weekdays, was put into the detector as written at $\delta = 0.3$, $k = 3$ with the two short widths: each of the last 28 days becomes 24 hourly sums, days are clustered into two or three groups and the grouping is kept when its median silhouette reaches 0.2, each weekday joins the group most of its days fell in, and the typical level is the median over the learned buckets of the same group within 30 minutes of the time of day over 28 days.

Series	False a month, plain	grouped	Drops to a tenth in 1 h, plain	grouped	Drops to 30% in 1 h, plain	grouped	Silhouette
Weekday/ weekend (modeled)	0.00	0.00	34%	100%	17%	54%	0.79
NYC taxi	0.00	0.18	95%	99%	18%	69%	0.70
Tweets FB	0.00	0.00	97%	99%	23%	50%	0.36
Tweets GOOG	0.00	0.00	95%	99%	19%	39%	0.42
Tweets AMZN	0.00	0.00	100%	100%	83%	85%	0.46
Tweets AAPL	0.00	0.00	89%	87%	34%	35%	0.34
Tweets KO	0.00	0.00	85%	82%	15%	15%	0.16
Tweets IBM	2.4	1.2	12%	29%	2%	1%	0.35
Tweets CRM	1.1	3.4	20%	32%	3%	3%	0.38
Tweets UPS	3.3	3.3	6%	8%	2%	1%	0.05
Tweets PFE	0.00	1.1	0%	1%	0%	0%	0.34
Daily, steady falling (modeled)	0.00	0.00	100%	100%	unchanged	unchanged	0.0, 0.60

Where the week has a shape the groups find it and the band tightens: every drop to a tenth on the weekday/weekend shape, and 3.8 times the drops to 30% within the hour on the taxi series and about twice on FB and GOOG, which have a weekly rhythm of their own. The cost falls on the thin tweet series, where the silhouette passes on noise at 0.34 to 0.38 and the grouped level is estimated from a third of the days: 3 more false alarms in 14 source-months, 2 of them on the thin tweets. The design leaves day groups out; grouping only where the gate admits the 30-minute width, which is where the silhouette can be trusted, is the obvious form and is untested.

6.8 Sender rules

Two sender-specific rules were replayed and set aside. A gap judged per time of day, alarming when a value has been quiet longer than it ever was at this time of day in the past week, detects

a steady sender's stop in 10 minutes with no false alarm and raises 455 to 1,750 false alarms per source-month on the modeled source of a hundred senders at a row a minute each, 5 to 17 per sender; the gap of Section 2.8 keeps the idea, drops the time of day, which left each hour a handful of spells to beat, and adds the margin and the ten-spell floor. A share band, each value's share of the source's rows against a pooled band of shares, detects every stop within the hour at a median of 40 minutes with $\delta = 0.4$, $k = 4$, misses halvings, and sees nothing the value's own series does not.

7 Discussion and limitations

The story of the introduction rests on one fact, on three levers, on a gate that follows the source's rows, and on judging in two coordinates, rows per unit of time and time per row. This section says what each of those rests on in turn, and where the ground is thin.

What the fact buys. Every band edge is an empirical quantile, so the per-reading false-alarm rate under exchangeability is known in advance and no tail is assumed. The constants beside the edges, the gate's ten rows, the margins δ and m and the ten-spell floor, were set by the replay. What the fact does not cover, dependence between windows and between readings, is paid for empirically by the margin and the wait, and the replay is the only measure of what they cost. The margin and wait were chosen on the real count series; the real logs were scored after the band was fixed, and their false-alarm rates (Section 4.2) are the out-of-sample measure this paper has for the band. The gap's margin was chosen with the logs in view (Section 4.7), so the logs are not out of sample for it.

Real logs are of two kinds, and the two coordinates divide them. Four of the ten logs have lines in nearly every bucket, and on them the count widths do what they do on the count series. The other six, an error log, a syslog, an authentication log, a supercomputer's events, a coordination service and a university web server at three lines a bucket, are judged by the count widths at the long widths or not at all, and where those judge they alarm, because a count of events that happen when something goes wrong has no normal rate; the same logs have a normal silence, and the gap judges them by it: 1.91, 1.02 and 0.96 alarms a month on the Apache error log, the Linux syslog and BGL where the count widths raise 6.14, 6.55 and 17.09 (Sections 4.2 and 4.7). The kinds describe the logs; the detector applies one rule, every width on every source where its gate admits it, and the operator sees one message. Section 4.8 scores the five widths together on every source.

The count widths are one estimator, and a fast one. The four count readings use the same typical window, the same ratio, the same quantile and the same wait; a width differs from the next only in what it sums. Their edge shares a bound with a rank-based CUSUM over the same pooled ratios: a reading below the design's edge, near the third-lowest of 2,000 ratios, is at most about 2.8 decades rare, and below the lowest 3.3, for either. The band is faster for two reasons. The margin is a value criterion past the extreme, so a reading far below the edge counts for more than the rarest rank; and the wait has no floor, so k readings merely below alarm, where the CUSUM subtracts a floor κ from every reading's evidence and at a quiet setting needs eleven empty readings to reach its bar.

A quantile needs windows that do not overlap. A week holds about 2,000 distinct half-hour windows and 7 distinct days. The 0.05% quantile of a 1-day width over a week is the worst day of the week, and a source 20% under its worst day is an alarm; over 28 days it is the worst of 28. The history behind a width should grow with the width; of the two histories tried, 7 and 28 days, 28 is the one that made the 1-day width quiet (Section 5.4). A width of a week would want months.

Halvings. No setting of this band catches most halvings within the hour on a variable series

without the baseline's alarm rate (Sections 6.1 and 6.2), and no width does (Section 6.5): a halving does not empty a long window in an hour. The 6-hour and 1-day widths catch 27% of halvings within 6 hours and 52% within the day on the real series, and that is the route to a halving this work found.

Bursty series. On one EC2 series a fifth of the 30-minute windows are bursts over ten times the median window (facts.py), at roughly repeating times of day. The detector as written learns the windows under an above-normal alarm, and catches 93% of the stops on that series, EC2 network 5abac7 of Section 4.1, within the hour and every one within the day; the early harness, which leaves those windows out, takes a burst as the typical level of its time of day and catches 27%. The BGL log is a burstier case still, an empty median bucket and bursts of up to 42,000 lines, and the band raises 17.09 false alarms a month on it, the most of any log: a series made of bursts has no level for a ratio to be taken against. The gap, which asks only how long the quiet between bursts has run, raises 0.96 a month there.

Weekly patterns. Day groups catch every drop to a tenth on the weekday/weekend shape and two to four times the drops to 30% on the real series with a weekly rhythm, and add false alarms on the thin series where the clustering passes on noise (Section 6.7). The band's alarms on the logs come mostly from the three sporadic system logs, which the gap judges better (Section 4.7); on the university web logs, which have a weekly and an academic rhythm both, it raises 69 alarms on Calgary and 14 on Saskatchewan over their records, so a grouped level gated like the widths is the first thing to try there.

Above normal is judged at 30 minutes only, the one width the replay scored it at.

Data. The count series and logs are public, not the application's own, and the sender mixes are modeled. A replay of a deployment's own 5-minute counts, per source and per sender, is the check these numbers need; the harness takes a timestamp,count CSV.

8 Application

In smeltr the five widths live in `src/leader/throughput.rs`, which judges every source from the 5-minute buckets the leader already keeps, at the quiet setting, $\delta = 0.3$ and $k = 4$, with no setting exposed: the three settings of Section 2.7 differ by one bucket on a stop and by five to nine times in false alarms, a choice the design makes once rather than one an operator should have to make. The leader has its own words for the paper's: the band is a source's range, the typical window its usual window, and an alarm a warning. The four count widths replace the baseline, learn from 28 days of buckets where it learned from 7, and the chart draws the band that is judged. The gap is a second statistic beside them, read from the same buckets: per source, the lengths of its completed spells over 28 days and the start of the current spell. Both are refreshed hourly; every width judges every source where its gate admits it, and a stop is reported by whichever fires first.

A below-normal alarm at a count width, once raised, ends on a reading inside the band or an operator's reset. While no width judges the source, the leader reads it against the normal it had before the alarm, so the alarm still says how far below it reads, and it ends when the source reads inside or above that normal. A gap alarm ends at the first row.

The leader does not judge or learn a window in which the source was blocked, refused, waiting or unreachable, since those stops are already reported. The replay had no such windows, so the application learns from the set of Section 2.5 less these.

The cost is small. A count reading builds a typical window and about 2,000 to 8,000 ratios and their quantile per width, about 200,000 floating-point operations against about 90 for the one band, a few milliseconds per source in Python and well under a second per 5-minute pass

for 250 sources in Rust. The gap adds a list of spell lengths per source, a few thousand integers at most, and one comparison per reading.

For senders, the Streams tally counts each value's rows for a watched field from the one batch per route per heartbeat it already samples, at most 256 values and their counts per field, about 1 KB. The leader selects the watched values by the rule of Section 2.6, the values seen every hour of a week of tallies, names them in the deployment and judges them at the count widths at the quiet setting; a 5-minute series per watched value is kept beside the one per source and dropped after a week's absence; a lowering names a source, a field and a value. The design rule that Streams never carries a value gains this one exception.

9 Reproducibility

The harness is Python 3 with NumPy, in `docs/research/normal-detector/lab/` beside this paper. Every run is seeded and was reproduced row for row; the fetched series and the runs' outputs are made by the commands below, and each comment names the sections the run feeds.

```
python3 fetch_nab.py nab/
    # the 14 NAB series, their labeled windows and license
sh fetch_logs.sh ../fetch
    # nine of the ten raw logs, unpacked under ../fetch/loghub/<log>/ and ../fetch/ita/
python3 real_logs.py ../fetch real/
    # 5-minute counts and fields from the ten logs (Sections 3 and 4.2)
python3 facts.py nab/ real/
    # the EC2 burst share and Apache's longest silences (Sections 4.2 and 7)
python3 extend_lab.py nab/ paper extend_paper.json
    # the three settings and the removable parts, real and modeled series
    # (Sections 4.1, 4.3 and 5.4)
python3 extend_lab.py logs paper extend_paper_logs.json
    # the same on the ten logs (Section 4.2)
python3 sparse_lab.py sparse_final.json
    # sparse sources (Section 4.4)
python3 sparse_lab.py sparse_history.json history
    # 7 and 28 days behind every width, and a week-wide width (Section 4.4)
python3 quiet_lab.py nab/ quiet.json
    # the gap at margins 1, 1.5 and 2 (Sections 2.8 and 4.7)
python3 either_lab.py nab/ either.json
    # count widths and gap scored together (Section 4.8)
python3 extend_lab.py nab/ hist7 extend_hist7.json
    # 7 days of history behind every width (Sections 4.4 and 5.4)
python3 extend_lab.py nab/ gate extend_gate.json
    # the gate at 5, 10, 20 and 40 rows (Section 5.4)
python3 extend_lab.py senders paper_senders extend_paper_values.json
    # each sender of each mix, default setting (Section 4.5)
python3 extend_lab.py senders paper_senders_quiet extend_paper_values_quiet.json
    # the same at the quiet setting (Section 4.5)
python3 values_logs.py real/ values_logs.json
    # the sender criterion and each watched value on the logs (Section 4.5)
python3 extend_lab.py nab/ widths extend_widths.json
    # each width alone (Section 4.6)
python3 extend_lab.py nab/ grid2 extend_grid2.json
    # margins x waits at the design (Section 5.2)
python3 heldout.py extend_grid2.json
    # pick on one half, score on the other (Section 5.2)
python3 baseline_drops.py nab/ baseline_drops.csv
    # the baseline on every drop fraction (Sections 1 and 4)
python3 balance.py sweep.csv
    # the precision model (Section 5.3)
python3 run_all.py nab/ final.csv && python3 aggregate.py final.csv
```

```
    # the twelve bands (Section 6.1)
python3 sweep.py nab/ sweep.csv        # 5 margins x 4 waits at one width (Section 6.2)
python3 patterns.py nab/ patterns.json

    # minutes from a stop to its alarm; what false alarms and misses look like (Section 6.1)
python3 rank_lab.py nab/ rank.json     # the rank-based CUSUM, 16 settings (Section 6.3)
python3 unified_lab.py nab/ unified.json  # widths between 5 and 30 minutes (Section 6.3)
python3 learn.py nab/ learn.json       # learning alarmed windows (Section 6.4)
python3 extend_lab.py nab/ long extend_long.json
    # a third width with 7 days of history (Section 6.5)
python3 extend_lab.py nab/ rows extend_rows.json &&
    python3 extend_lab.py nab/ rowsfloor extend_rowsfloor.json
    # widths defined in rows (Section 6.6)
python3 extend_lab.py nab/ groups extend_groups.json
    # day groups in the detector as written (Section 6.7)
python3 share_lab.py share.json && python3 silence_lab.py silence.json
    # the share band and the time-of-day gap (Section 6.8)
python3 discover_lab.py discover.json
    # which modeled fields behave like senders (Section 4.5)
python3 build_paper.py . paper_v7_template.md paper.md
    # every table and quoted number in this paper, from the files above
```

`design_lab.py` is the two-width detector as one class; `extend_lab.py` adds to it, one rule at a time, the long widths, the history per width, the gate, the typical window, the day groups and the rows-defined widths, and the `paper` experiment is the detector as recommended beside the parts removed one at a time; `normal_lab.py` is the early harness: it ports the baseline and holds every candidate band and the scoring; `run_all.py` the modeled series and the NAB loader; `real_logs.py` the log parsers. The tables of Sections 6.1 and 6.5 to 6.8 and the numbers quoted beside them are copied into the template by hand from the outputs of the runs named for them (`aggregate.py` on `final.csv`, `extend_long.json`, `extend_rows.json`, `extend_rowsfloor.json`, `extend_groups.json`, `patterns.json`, `unified.json`, `share.json`, `silence.json`); every other table and quoted number is written by `build_paper.py` from the files above, and the template names the file and field each comes from. The Google cluster 2011 job-events tables are Google's own distribution, placed under `../fetch/gcluster/`. `md2tex.py` sets the paper in LaTeX, `gen_diagram.py` and `diagram_code.js` draw Figure 1, and `run_chain.sh` runs the long steps so that each resumes where it stopped; `confidence_lab.py`, `five_lab.py`, `multi_lab.py`, `tail_lab.py` and `value_lab.py` are earlier experiments on the early harness, kept for the record, whose outputs the paper does not quote. A real source exported as a timestamp,count CSV joins the replay as one more series, and with no labeled windows every alarm on it counts as false.

Appendix A. The distribution-free bound and the three levers

Proposition 1. *Let $X_1, \ldots, X_{n+1}$ be exchangeable real random variables with no ties almost surely. Then*

$$P\left(X_{n+1} < \min(X_1, \ldots, X_n)\right) = \frac{1}{n+1}.$$

Proof. The event is that X_{n+1} is the minimum of the n+1 variables. By exchangeability the joint law is invariant under permutations of the indices, so each index is the minimum with the same probability, and with no ties exactly one of them is. The n+1 probabilities sum to one. □

Corollary 2. *Let $X_{(1)} \le \cdots \le X_{(n)}$ be the order statistics of $X_1, \ldots, X_n$ (David and Nagaraja, 2003) and let $r \in \{1, \ldots, n\}$. Then $P(X_{n+1} < X_{(r)}) = r/(n+1)$. With ties, $X_{n+1} < X_{(r)}$ implies*

that X_{n+1} has rank at most r under any tie-breaking, so $P(X_{n+1} < X_{(r)}) \le r/(n+1)$; counts tie, and the inequality is the form used. A band whose lower edge is interpolated between $X_{(r)}$ and $X_{(r+1)}$ is undercut by an exchangeable next window with probability at most $(r+1)/(n+1)$.

Proof. Without ties, $X_{n+1} < X_{(r)}$ if and only if X_{n+1} has rank at most r among the n+1, and by exchangeability each rank is equally likely. □

This is the fact behind conformal prediction (Vovk, Gammerman and Shafer, 2005). It makes no assumption on the law of the windows, so the heavy tails of real count series cost nothing. It does assume exchangeability, which fails in two ways for windows of a count series.

Time of day. Windows at 3 a.m. and 3 p.m. are not drawn the same way on any series with a daily pattern. Pooling them raw destroys the bound's premise and, worse, destroys detection: the band's low edge is set by the quietest hour, and a daytime drop to the night level is inside it. The remedy is to divide each window by its time of day's typical level before pooling (Section 2.2).

Overlap and drift. Consecutive windows of width w share $w-1$ buckets, and the noise of a real series drifts over tens of minutes. The windows of a week are therefore far from independent, the effective n is much smaller than the count of windows, and consecutive readings are dependent: a window that sits at the band's edge produces a run of readings in and out of the band. Proposition 1 bounds the rate of readings below the band; it does not bound the rate of alarms raised, which under flicker is several per excursion. The wider the window, the fewer windows of a given history do not overlap: a week holds 7 windows of a day, and the bound's n for that width is nearer 7 than 2,000, which is why the long widths take 28 days (Section 2.2).

Three levers follow. More exchangeable windows lower $r/(n+1)$ at a fixed rank: an edge at the lowest of 91 windows is undercut once in 92 readings; the design's edge, interpolated between the second- and third-lowest of about 2,000, at most 3 times in 2,017, once in about 670, a factor of seven. A width judged every five minutes makes 8,640 readings a month, so even that edge is undercut about 13 times a month under exchangeability alone, and a quantile by itself cannot reach one alarm a month. A margin below $X_{(r)}$ requires the next window to fall further than any past one did; the fact does not price it, and what it buys depends on the tails of the series. A wait of k readings alarms only on the k-th consecutive reading below the band, so an excursion pays once, and a flicker shorter than k readings pays nothing. The detector pulls all three (Section 2); what each buys is measured in Section 6.1.

References

- Arlitt, M. F. and Williamson, C. L. (1996). Web server workload characterization: the search for invariants. *Proceedings of ACM SIGMETRICS 1996*, 126–137. https://researchr.org/publication/ArlittW96. The logs are in the Internet Traffic Archive, https://ita.ee.lbl.gov/html/traces.html
- David, H. A. and Nagaraja, H. N. (2003). *Order Statistics*, 3rd ed. Wiley. https://doi.org/10.1002/0471722162
- He, S., Zhu, J., He, P. and Lyu, M. R. (2020). Loghub: a large collection of system log datasets towards automated log analytics. arXiv:2008.06448. https://arxiv.org/abs/2008.06448. The logs are at https://github.com/logpai/loghub
- Lavin, A. and Ahmad, S. (2015). Evaluating real-time anomaly detection algorithms: the Numenta Anomaly Benchmark. *IEEE ICMLA*. https://arxiv.org/abs/1510.03336

- Page, E. S. (1954). Continuous inspection schemes. *Biometrika* 41, 100–115. https://doi.org/10.1093/biomet/41.1-2.100
- Rajyaguru, O., Bai, H. and Curtis, K. (2023). ML-powered assistance for adaptive thresholding in ITSI. Splunk blog, 9 August 2023. https://www.splunk.com/en_us/blog/it/ml-powered-assistance-for-adaptive-thresholding-in-itsi.html
- Reiss, C., Wilkes, J. and Hellerstein, J. L. (2011). Google cluster-usage traces: format + schema. Google Inc. technical report. https://github.com/google/cluster-data
- Vovk, V., Gammerman, A. and Shafer, G. (2005). *Algorithmic Learning in a Random World*. Springer. https://doi.org/10.1007/b106715